JCL & Z/OS

CHEAT SHEETS

Job Control Language and the z/OS basics around it

z/OS · Datasets · ISPF · Zowe · JOB · EXEC · DD · DISP · SPACE · DCB · STEPLIB · Temp Datasets · COND · IF / THEN · Procs · Symbols · IEFBR14 · IEBGENER · IEBCOPY · IDCAMS · DFSORT · IKJEFT01 · Compile & Link · GDGs · JES2 · Abends

32 REFERENCE SHEETS

CONTENTS

Z/OS BASICS

01 Mainframe & z/OS Overview
02 Datasets
03 TSO / ISPF Essentials
04 Modern Tools: Zowe & z/OSMF

JCL FUNDAMENTALS

05 JCL Syntax
06 The JOB Statement
07 The EXEC Statement
08 DD Statement Basics

DD STATEMENTS IN DEPTH

09 The DISP Parameter
10 SPACE, UNIT & Volumes
11 DCB Attributes
12 Special DD Names
13 Concatenation, Temp Datasets & Backward References

CONDITIONAL PROCESSING

14 Return Codes & COND
15 IF / THEN / ELSE & Restart

PROCEDURES & SYMBOLS

16 In-Stream & Cataloged Procedures
17 Symbolic Parameters & SET
18 INCLUDE & Overrides

UTILITIES

19 IEFBR14 & IEBGENER
20 IEBCOPY
21 IDCAMS
22 DFSORT
23 TSO in Batch & Other Utilities

RUNNING COBOL & MANAGING DATA

24 Compile, Link & Go
25 Running a COBOL Batch Program
26 Generation Data Groups (GDGs)

JOB OUTPUT, DEBUGGING & SCHEDULING

27 JES2 & Job Output
28 Reading Job Logs & JCL Errors
29 Common Abends
30 Scheduling & Modern Workflow

QUICK REFERENCE

31 Quick Reference — Statements & Parameters
32 Quick Reference — Utilities, RCs & Abends

JCL

Table of Contents

32 sheets · syntax · examples · notes

Z/OS BASICS 01 – 04

- 01 Mainframe & z/OS Ove…
- 02 Datasets
- 03 TSO / ISPF Essentials
- 04 Modern Tools: Zowe & …

JCL FUNDAMENTALS 05 – 08

- 05 JCL Syntax
- 06 The JOB Statement
- 07 The EXEC Statement
- 08 DD Statement Basics

DD STATEMENTS IN DEPTH 09 – 13

- 09 The DISP Parameter
- 10 SPACE, UNIT & Volumes
- 11 DCB Attributes
- 12 Special DD Names
- 13 Concatenation, Temp …

CONDITIONAL PROCESSING 14 – 15

- 14 Return Codes & COND
- 15 IF / THEN / ELSE & Rest…

PROCEDURES & SYMBOLS 16 – 18

- 16 In-Stream & Cataloge…
- 17 Symbolic Parameters …
- 18 INCLUDE & Overrides

UTILITIES 19 – 23

- 19 IEFBR14 & IEBGENER
- 20 IEBCOPY
- 21 IDCAMS
- 22 DFSORT
- 23 TSO in Batch & Other …

RUNNING COBOL & MANAGING DATA 24 – 26

- 24 Compile, Link & Go
- 25 Running a COBOL Batc…
- 26 Generation Data Grou…

JOB OUTPUT, DEBUGGING & SCHEDULING 27 – 30

- 27 JES2 & Job Output
- 28 Reading Job Logs & JC…
- 29 Common Abends
- 30 Scheduling & Modern …

QUICK REFERENCE 31 – 32

- 31 Quick Reference — Sta…
- 32 Quick Reference — Util…

Sheet 01

Mainframe & z/OS Overview

IBM Z · z/OS · LPARs · batch vs online · JES2 · subsystems (DB2, CICS, IMS, MQ) · EBCDIC

THE PLATFORM BEHIND THE JOBS

IBM Z mainframes run **z/OS**, the operating system behind most bank, insurance and government batch processing. Work runs either **online** (CICS or IMS transactions, answered in milliseconds) or as **batch jobs**: JCL scripts submitted to **JES2**, which queues them, runs each step and collects the output.

KEY POINTS

- One machine is split into **LPARs**, each running its own OS image
- JCL tells z/OS which programs to run and which datasets to give them
- Text is **EBCDIC**, not ASCII: `A` is X'C1', a space is X'40'
- Batch jobs usually run overnight in a **batch window**

YOUR FIRST JOB

```
//HELLO    JOB (ACCT),'FIRST JOB',CLASS=A,MSGCLASS=X,
//         NOTIFY=&SYSUID
//* Print a message with the IEBGENER copy utility
//STEP1    EXEC PGM=IEBGENER
//SYSPRINT DD SYSOUT=*
//SYSIN    DD DUMMY
//SYSUT2   DD SYSOUT=*
//SYSUT1   DD *
HELLO FROM Z/OS
/*
```

CODE EXPLAINED

→ The JOB statement names the job and how JES should treat it

→ One EXEC step runs the IEBGENER program

→ SYSUT1 (input) is in-stream data; SYSUT2 (output) goes to the job log

→ Submit it, then read the output in SDSF or Zowe Explorer

MODERN EQUIVALENT

↔ **Python** `subprocess.run(["cat"], input="HELLO")` in a scheduled script

↔ **Java** `ProcessBuilder("cat")` launched by a scheduler

↔ **JavaScript** `child_process.spawn("cat")` from a cron-triggered Node script

ABEND CHECK

✖ `JCL ERROR` with no step run: a syntax problem, check the JESYSMSG listing

✖ Job sits in the input queue: wrong CLASS or held (`TYPRUN=HOLD`)

KEY TERMS

TERM	MEANING
Job	Unit of batch work (JOB statement)
Step	One program run (EXEC)
Dataset	A z/OS file
DD	Connects a dataset to a step
JES2	Job entry: queue, run, spool
Spool	Where job output is held
SYSOUT	Output sent to the spool

SUBSYSTEMS

NAME	ROLE
CICS	Online transactions
IMS	Hierarchical DB + transactions
DB2	Relational database
MQ	Messaging
RACF	Security
USS	UNIX shell on z/OS

⚠ COMMON MISTAKES

- Expecting files and folders: z/OS has datasets with dotted names
- Assuming ASCII when comparing or sorting data (EBCDIC sorts letters before digits)
- Submitting without NOTIFY and never seeing that the job finished

✓ TIPS & BEST PRACTICES

- Get free z/OS access through IBM Z Xplore to practice
- Learn to read the job log before writing complex JCL
- Keep each job's JCL in a PDS member under version control

QUICK SUMMARY

- z/OS — the mainframe OS
- JOB → steps → programs
- JES2 — queues and spools jobs
- EBCDIC — mainframe encoding

Datasets

Z/OS FILES

A **dataset** is a z/OS file with a name of up to 44 characters made of dot-separated **qualifiers**. **Sequential** (PS) datasets hold records in order; **partitioned** datasets (PDS/PDSE) hold named **members**, like files in a folder. Each dataset has fixed attributes: record format, record length and block size.

KEY POINTS

- Each qualifier: 1–8 characters, starting with a letter or `@ # $`
- The first qualifier (HLQ) is usually your user ID or an application
- Members are named in parentheses: `USER.JCL(PAYROLL)`
- Datasets are found through the **catalog**, so you rarely give a volume

ALLOCATE A PS AND A PDSE

```
//ALLOC    JOB (ACCT),'ALLOCATE',CLASS=A,MSGCLASS=X
//STEP1    EXEC PGM=IEFBR14
//* Sequential dataset: 80-byte fixed-blocked records
//SEQ      DD DSN=USER.TEST.DATA,DISP=(NEW,CATLG,DELETE),
//            SPACE=(TRK,(5,5)),
//            RECFM=FB,LRECL=80,BLKSIZE=0
//* Library (PDSE) for JCL members
//LIB      DD DSN=USER.JCL,DISP=(NEW,CATLG,DELETE),
//            SPACE=(TRK,(10,10)),DSNTYPE=LIBRARY,
//            RECFM=FB,LRECL=80
```

CODE EXPLAINED

- → IEFBR14 does nothing: the DD statements create the datasets
- → `CATLG` adds each new dataset to the catalog
- → `BLKSIZE=0` lets z/OS choose the best block size
- → `DSNTYPE=LIBRARY` makes a PDSE instead of a PDS

MODERN EQUIVALENT

- ↔ **Python** `pathlib.Path("test/data.txt").touch()` ; folders hold files
- ↔ **Java** `Files.createFile(Path.of("test/data.txt"))`
- ↔ **JavaScript** `fs.writeFileSync("test/data.txt", "")`

ABEND CHECK

- ✖ `IEF253I DUPLICATE NAME ON DIRECT ACCESS VOLUME` : the dataset already exists; delete it or use another name
- ✖ Qualifier over 8 characters: `JCL ERROR` , invalid DSN

DATASET TYPES

TYPE	HOLDS
PS	Sequential records
PDS	Members; needs compress
PDSE	Members; reuses space
VSAM KSDS / ESDS / RRDS	Keyed / entry / relative
GDG	Generations of a dataset
zFS / HFS	UNIX file systems

ATTRIBUTES

ATTRIBUTE	MEANING
RECFM=FB	Fixed, blocked
RECFM=VB	Variable, blocked
LRECL	Record length
BLKSIZE	Block size (0 = system)
DSORG=PS / PO	Sequential / partitioned

⚠ COMMON MISTAKES

- Lowercase or too-long qualifiers
- Writing to a PDS until it runs out of directory blocks (use PDSE)
- Creating a dataset that already exists (not cataloged twice)

✓ TIPS & BEST PRACTICES

- Prefix datasets with your user ID while testing
- Use PDSE (`DSNTYPE=LIBRARY`) for new libraries
- Let SMS and `BLKSIZE=0` pick placement and block size

QUICK SUMMARY

- `HLQ.MLQ.LLQ` — dataset name
- PS — sequential
- PDS/PDSE — members
- RECFM / LRECL — record shape

Sheet 03

TSO / ISPF Essentials

ISPF primary menu · 3.4 DSLIST · ISPF editor commands · SUBMIT · SDSF (ST, O, ?) · TSO commands

THE CLASSIC GREEN SCREEN

TSO is the interactive command line on z/OS, and **ISPF** is the menu-driven interface on top of it. You browse datasets in option **3.4**, edit members in the ISPF editor, type **SUB** to submit JCL, and check results in **SDSF**. Many developers now use Zowe Explorer instead, but ISPF is still everywhere.

KEY POINTS

- Primary commands go on the Command line; line commands in the line-number area
- PF3 saves and exits; `CANCEL` exits without saving
- `=3.4` jumps straight to the dataset list from anywhere
- SDSF `ST` lists your jobs; `?` shows a job's output files

EVERYDAY COMMANDS

```
=3.4                  Dataset list (DSLIST)
  Dsname Level: USER.*
  E  next to a member    Edit       B  Browse      V  View
  Z                      Compress a PDS

ISPF editor primary commands
  SUB                    Submit this JCL
  F PAYROLL ALL          Find every PAYROLL
  C OLD NEW ALL          Change all OLD to NEW
  HILITE JCL             Color JCL keywords
  RES                    Clear messages
  CANCEL                 Exit without saving

Line commands (type over the line numbers)
  I5  insert 5 lines   D  delete   DD/DD  delete block
  R   repeat           C/A  copy after   M/B  move before

=SD ; ST              SDSF status of your jobs
  ?  job's spool files   S  view   P  purge
```

CODE EXPLAINED

→ Commands are typed on the screen, not run as a script

→ Line commands act on the lines they are typed on

→ `HILITE JCL` colors JCL syntax in the editor

→ In SDSF, `S` next to JESMSGLG shows the job log

MODERN EQUIVALENT

↔ **Python** Terminal: `ls`, `vim`, then run `python job.py` and read stdout

↔ **Java** An IDE (IntelliJ) plus a Run console

↔ **JavaScript** VS Code plus the integrated terminal

ABEND CHECK

✖ `INVALID COMMAND` in ISPF: you typed a TSO command; prefix it with `TSO`

✖ Edit fails with "member in use": someone else (or another session) has it open

ISPF OPTIONS

OPTION	PANEL
=2	Edit
=3.2	Allocate / delete datasets
=3.3	Move / copy
=3.4	Dataset list
=6	TSO command shell
=SD	SDSF (site-dependent)

TSO COMMANDS

COMMAND	DOES
LISTCAT LEVEL(USER)	List datasets
ALLOCATE / FREE	Allocate DDs
SUBMIT 'USER.JCL(X)'	Submit job
STATUS	Job status
DELETE 'USER.X'	Delete dataset

⚠ COMMON MISTAKES

- Pressing PF3 when you meant to throw away changes (it saves)
- Purging job output before reading the error messages
- Forgetting quotes around full dataset names in TSO (your prefix gets added)

✓ TIPS & BEST PRACTICES

- Learn `=3.4`, `SUB`, `=SD ; ST` and `?` first
- Turn on `HILITE JCL` to spot typos
- Use Zowe Explorer for editing, ISPF for quick checks

QUICK SUMMARY

- `=3.4` — dataset list
- `SUB` — submit JCL
- SDSF `ST` / `?` — job output
- line commands — edit lines

Sheet 04

Modern Tools: Zowe & z/OSMF

Zowe CLI `zowe zos-files` / `zowe zos-jobs submit` · Zowe Explorer in VS Code · profiles & `zowe config` · z/OSMF · USS shell · IBM Z Xplore

WORKING FROM A LAPTOP

Zowe is the open-source framework for z/OS. **Zowe CLI** lets you list datasets, upload source, submit jobs and download output from any terminal or CI pipeline. **Zowe Explorer** does the same inside VS Code. Both talk to **z/OSMF**, the REST API layer on z/OS.

KEY POINTS

- Install with `npm install -g @zowe/cli`
- `zowe config init` creates `zowe.config.json` with your host and user
- Command groups: `zos-files` (`files`), `zos-jobs` (`jobs`), `zos-tso` , `zos-uss`
- USS gives a real UNIX shell on z/OS via SSH

ZOWE CLI SESSION

```
npm install -g @zowe/cli
zowe config init                        # host, port, user

zowe files list ds "USER.*"             # list datasets
zowe files list am "USER.JCL"           # list members
zowe files download am "USER.JCL" -d jcl/
zowe files upload ftds jcl/PAYROLL.jcl "USER.JCL(PAYROLL)"

zowe jobs submit ds "USER.JCL(PAYROLL)" --wait-for-output
# jobid: JOB04711  retcode: CC 0000  status: OUTPUT
zowe jobs list sfbi JOB04711            # spool files
zowe jobs view sfbi JOB04711 2          # JESMSGLG
zowe jobs download output JOB04711 -d out/
zowe jobs delete job JOB04711
```

CODE EXPLAINED

→ `--wait-for-output` blocks until the job ends and prints the return code

→ Spool file IDs come from `list sfbi` (spool files by job ID)

→ Download output to check it into tickets or test reports

→ The same commands run in GitHub Actions or Jenkins

MODERN EQUIVALENT

↔ **Python** `subprocess.run(["zowe", "jobs", "submit", …])` or the Zowe Python SDK

↔ **Java** Zowe Java SDK (`zowe-client-java-sdk`)

↔ **JavaScript** Zowe Node.js SDK (`@zowe/zos-jobs-for-zowe-sdk`)

ABEND CHECK

✖ `401 Unauthorized` from z/OSMF: wrong password or expired; update the profile

✖ `ECONNREFUSED` / TLS errors: wrong port or self-signed cert (`rejectUnauthorized: false` only for labs)

COMMON COMMANDS

COMMAND	DOES
`files list ds "HLQ.*"`	List datasets
`files download am "DS"`	Download all members
`files upload dtp dir "DS"`	Upload a folder to a PDS
`jobs submit ds "DS(M)"`	Submit JCL
`jobs submit lf file.jcl`	Submit a local file
`jobs view jsbj JOBID`	Job status
`tso issue cmd "LISTCAT"`	Run a TSO command

TOOLS

TOOL	USE
Zowe CLI	Terminal, scripts, CI
Zowe Explorer	VS Code datasets & jobs
IBM Z Open Editor	COBOL / JCL language support
z/OSMF	REST APIs + web UI
IBM Z Xplore	Free learning system

⚠ COMMON MISTAKES

- Storing passwords in plain text config files committed to Git
- Submitting from CI without waiting for and checking the return code
- Uploading files with Windows line endings or tabs

✓ TIPS & BEST PRACTICES

- Use `zowe config secure` to store credentials in the OS keychain
- Always use `--wait-for-output` in pipelines and fail on RC > 4
- Keep JCL as `.jcl` files in Git and upload on build

QUICK SUMMARY

- Zowe CLI — z/OS from a terminal
- Zowe Explorer — VS Code
- z/OSMF — REST layer
- `--wait-for-output` — get the RC

Sheet 05

JCL Syntax

// in columns 1–2 · name, operation, operand & comment fields · column 72 limit · continuation rules · //* comments · /* delimiter · // null statement

HOW A JCL LINE IS BUILT

Every JCL statement starts with // in columns 1–2, followed by a **name** (starting in column 3), an **operation** (JOB, EXEC, DD), **operands** separated by commas with no spaces, and optional comments. Statements must end by column 71; to continue, end the line after a comma and resume between columns 4 and 16.

KEY POINTS

- Names: 1–8 characters, start with a letter or `@ # $`
- Spaces separate fields; a space ends the operands
- JCL is uppercase (except inside quotes and in-stream data)
- Column 72 non-blank used to mean continuation (rare today)

FIELDS AND CONTINUATION

```
//*-+----1----+----2----+----3----+----4----+----5----+----6----+-
//* name   operation  operands                        comments
//PAYROLL  JOB  (ACCT),'R BREUTZMANN',CLASS=A,
//         MSGCLASS=X,NOTIFY=&SYSUID
//STEP1    EXEC PGM=PAYCALC,PARM='2026-11'  RUN FOR NOV
//INFILE   DD   DSN=PROD.PAY.TRANS,
//              DISP=SHR
//OUTFILE  DD   SYSOUT=*
//SYSIN    DD   *
DATA LINES HAVE NO // AND ARE NOT JCL
/*
//
```

CODE EXPLAINED

→ Ruler: `//` fills columns 1–2, names start in column 3

→ A trailing comma means "continued on the next line"

→ Text after the first space following operands is a comment

→ `/*` ends in-stream data; a lone `//` marks the end of the job

MODERN EQUIVALENT

↔ **Python** Python: indentation and newlines are syntax; `\` or brackets continue lines

↔ **Java** Java: free format; statements end with `;`

↔ **JavaScript** JavaScript: free format; YAML-like configs (e.g. GitHub Actions) are the closer analog

ABEND CHECK

✖ `IEFC605I UNIDENTIFIED OPERATION FIELD`: the name or operation is misspelled or misaligned

✖ `EXPECTED CONTINUATION NOT RECEIVED`: a line ended with a comma but the next line starts in the wrong column

STATEMENT TYPES

STATEMENT	PURPOSE
`//name JOB`	Start a job
`//name EXEC`	Start a step
`//name DD`	Define a dataset
`//* text`	Comment
`/*`	End of in-stream data
`//`	End of job (null statement)
`// IF / ENDIF`	Conditional steps

COLUMN RULES

COLUMNS	USE
`1-2`	//
`3-10`	Name (optional on DD continuations)
`After name`	One space, then operation
`4-16`	Continuation operands start
`72`	Old continuation marker
`73-80`	Ignored (sequence)

⚠ COMMON MISTAKES

- A space inside the operand list, which turns the rest into a comment
- Continuing a line that does not end with a comma
- Starting a continuation in column 17 or later

✓ TIPS & BEST PRACTICES

- Use `HILITE JCL` in ISPF or IBM Z Open Editor in VS Code
- Put one parameter group per continuation line for readability
- Run `TYPRUN=SCAN` to check syntax without running anything

QUICK SUMMARY

- `//name op operands` — every statement
- comma at end — continue
- space — ends operands
- `//*` comment · `/*` data end

The JOB Statement

job name · accounting & programmer name · CLASS · MSGCLASS · MSGLEVEL=(1,1) · NOTIFY=&SYSUID · REGION / TIME · TYPRUN=SCAN / HOLD

STARTING EVERY JOB

The **JOB** statement is the first line of every job. It gives the job a name, accounting information and a programmer name, then tells JES2 how to run it: which **class** (queue) to use, where to send messages (MSGCLASS), how much detail to list and who to notify when it ends.

KEY POINTS

- Positional first: accounting, then programmer name
- Keyword parameters follow in any order
- Job names are often your user ID plus a letter
- Shops publish which CLASS and MSGCLASS values to use

JOB STATEMENT VARIATIONS

```
//USER01A  JOB (D1234),'PAYROLL TEST',
//         CLASS=A,
//         MSGCLASS=X,
//         MSGLEVEL=(1,1),
//         NOTIFY=&SYSUID,
//         REGION=0M,
//         TIME=(5,0)
//*
//* Check syntax only: nothing runs
//USER01S  JOB (D1234),'SCAN',CLASS=A,MSGCLASS=X,
//         TYPRUN=SCAN
//*
//* Hold in the queue until released in SDSF (A)
//USER01H  JOB (D1234),'HELD',CLASS=A,MSGCLASS=X,
//         TYPRUN=HOLD
```

CODE EXPLAINED

- → `MSGLEVEL=(1,1)` : list all JCL, procs and allocation messages
- → `NOTIFY=&SYSUID` sends a TSO message to whoever submitted it
- → `REGION=0M` asks for the maximum memory the site allows
- → `TIME=(5,0)` limits the whole job to 5 CPU minutes

MODERN EQUIVALENT

- ↔ **Python** A job definition: name, queue and timeout, like a CI job or Celery task options
- ↔ **Java** Job config for a scheduler (Spring Batch job parameters, Quartz)
- ↔ **JavaScript** A GitHub Actions `job:` with `runs-on` and `timeout-minutes`

ABEND CHECK

- ✖ `S322` abend: the TIME limit was exceeded; raise TIME or fix a loop
- ✖ `IEFC452I JOB NOT RUN - JCL ERROR` with a bad CLASS: use a class your site defines

JOB PARAMETERS

PARAMETER	MEANING
(acct)	Accounting info (site format)
'name'	Programmer name, max 20
CLASS=	Input queue / job class
MSGCLASS=	Output class for the job log
MSGLEVEL=(s,m)	JCL / message detail
NOTIFY=	User to notify
REGION=	Memory per step
TIME=	CPU time limit
TYPRUN=	SCAN, HOLD, COPY
RESTART=	Rerun from a step

MSGLEVEL

VALUE	LISTS
(0,0)	JOB statement only
(1,1)	All JCL + all messages
(2,0)	Your JCL only
(1,0)	All JCL, messages on failure

⚠ COMMON MISTAKES

- A job name already in use by another running job (it waits)
- Missing NOTIFY, so you poll SDSF waiting for results
- Copying a production JOB card with production accounting into test

✓ TIPS & BEST PRACTICES

- Keep a personal JOB card member and `COPY` it into new JCL
- Use `TYPRUN=SCAN` before running new or edited jobs
- Ask which MSGCLASS keeps output held so you can read it

QUICK SUMMARY

- `JOB` — first statement
- `CLASS` — queue
- `MSGCLASS` / `MSGLEVEL` — log
- `TYPRUN=SCAN` — syntax check

Sheet 07

The EXEC Statement

step names · PGM= · PROC= (or proc name) · PARM= · REGION / TIME on steps · step limits · PARMDD

RUNNING A PROGRAM OR PROCEDURE

Each **EXEC** statement starts a **step**. PGM= runs a load module found in STEPLIB, JOBLIB or the system libraries. Naming a procedure runs all its steps instead. PARM= passes up to 100 characters to the program, and step-level REGION, TIME and COND override job defaults.

KEY POINTS

- Step names: 1–8 characters, unique within the job
- A job can have up to 255 steps
- `PARM` with commas or spaces must be in quotes
- `PARMDD` points to a DD with a longer parameter (up to 32,760 bytes)

EXEC FORMS

```
//STEP010  EXEC PGM=PAYCALC
//*
//STEP020  EXEC PGM=PAYRPT,PARM='2026-11,SUMMARY'
//*
//STEP030  EXEC PGM=BIGJOB,REGION=512M,TIME=(10,0)
//*
//* Run a cataloged procedure (all its steps)
//COMPILE  EXEC IGYWCL
//* or explicitly:
//COMPILE2 EXEC PROC=IGYWCL
//*
//* Long parameters from a DD
//STEP040  EXEC PGM=EXTRACT,PARMDD=MYPARM
//MYPARM   DD *
REGION=MIDWEST,START=20261101,END=20261130,FORMAT=CSV
/*
```

CODE EXPLAINED

- → STEP010 runs PAYCALC with job defaults
- → The program reads PARM through its LINKAGE SECTION
- → Step REGION/TIME override the JOB values for that step only
- → Naming a proc without `PGM=` runs the procedure

MODERN EQUIVALENT

- ↔ **Python** `python paycalc.py 2026-11 SUMMARY` → `sys.argv[1:]`
- ↔ **Java** `java PayRpt 2026-11 SUMMARY` → `String[] args`
- ↔ **JavaScript** `node payrpt.js 2026-11 SUMMARY` → `process.argv.slice(2)`

ABEND CHECK

- ✖ `S806` abend: the program was not found; check STEPLIB/JOBLIB and the name
- ✖ `IEFC642I EXCESSIVE PARAMETER LENGTH` on PARM: over 100 characters, use PARMDD

EXEC PARAMETERS

PARAMETER	MEANING
PGM=name	Load module to run
PROC=name / name	Procedure to run
PARM='…'	Up to 100 chars to program
PARMDD=ddname	Long PARM from a DD
REGION=	Memory for the step
TIME=	CPU limit for the step
COND=	Skip test (sheet 14)

PROGRAM SEARCH ORDER

WHERE	NOTES
STEPLIB	This step only
JOBLIB	All steps (if no STEPLIB)
LPA	Link pack area
LINKLIST	System libraries

⚠ COMMON MISTAKES

- Duplicate step names, which breaks referbacks and restarts
- Unquoted PARM containing commas
- Forgetting STEPLIB for programs in your own load library

✓ TIPS & BEST PRACTICES

- Number steps by 10s (STEP010, STEP020) so you can insert steps
- Use descriptive step names: `SORTIN`, `UPDATE`, `REPORT`
- Put long or changing parameters in a PARMDD member

QUICK SUMMARY

- `EXEC PGM=` — run a program
- `EXEC procname` — run a proc
- `PARM=` — up to 100 chars
- STEPLIB — where to find it

DD Statement Basics

CONNECTING DATA TO A STEP

A **DD** (data definition) statement connects a name the program uses to a real dataset, a print destination or in-stream data. The program never names the dataset: it opens a DD name, and the JCL decides what that is. This is why the same program can run against test and production data.

KEY POINTS

- DD names: 1–8 characters, must match the program's file names
- `SYSOUT=*` sends output to the job log (MSGCLASS)
- `DUMMY` gives an empty input or throws output away
- `DD *` ends at `/*` or the next `//`; `DD DATA` only at `/*` (or DLM)

IN COBOL

- `SELECT CUST-FILE ASSIGN TO CUSTIN` opens the DD named CUSTIN
- `ACCEPT` with no FROM reads the SYSIN DD
- `DISPLAY` writes to SYSOUT (or the DD named in the `OUTDD` option)

COMMON DD TYPES

```
//STEP1    EXEC PGM=CUSTUPD
//* Existing cataloged dataset, shared read
//CUSTIN   DD DSN=PROD.CUSTOMER.MASTER,DISP=SHR
//* New output dataset
//CUSTOUT  DD DSN=USER.CUSTOMER.NEW,DISP=(NEW,CATLG,DELETE),
//            SPACE=(CYL,(5,2),RLSE),RECFM=FB,LRECL=200
//* Print output to the job log
//RPTOUT   DD SYSOUT=*
//* No input this run
//TRANIN   DD DUMMY
//* In-stream control cards (read by ACCEPT)
//SYSIN    DD *
RUNDATE=20261112
/*
//* In-stream data that contains // lines
//JCLCOPY  DD DATA,DLM=$$
//OLDJOB   JOB (ACCT),'COPIED'
$$
```

CODE EXPLAINED

- → The program opens CUSTIN; the JCL picks the master file
- → CUSTOUT is created, cataloged on success and deleted on abend
- → TRANIN DUMMY: the first READ gets end of file
- → `DLM=$$` lets data contain `//` lines

MODERN EQUIVALENT

- ↔ **Python** `open(os.environ["CUSTIN"])`: config decides the file, not the code
- ↔ **Java** `new FileReader(System.getenv("CUSTIN"))`
- ↔ **JavaScript** `fs.createReadStream(process.env.CUSTIN)`

ABEND CHECK

- ✖ `IEC130I CUSTIN DD STATEMENT MISSING` / FILE STATUS 35: the DD name is missing or misspelled
- ✖ `IEF212I … DATA SET NOT FOUND`: wrong DSN or not cataloged

DD FORMS

FORM	USE
`DSN=x,DISP=SHR`	Read existing
`DSN=x,DISP=OLD`	Exclusive use
`DSN=x,DISP=(NEW,CATLG,DELETE)`	Create
`SYSOUT=*`	Job log / printer
`DUMMY`	Empty input / discard
`DD *`	In-stream data
`DD DATA,DLM=xx`	Data with // lines
`PATH='/u/…'`	UNIX file

⚠ COMMON MISTAKES

- DD name in the JCL not matching the ASSIGN name in the program
- Using `DD *` for data that starts with `//` (use DD DATA)
- DISP=OLD on a shared production file, locking other jobs out

✓ TIPS & BEST PRACTICES

- Use DISP=SHR for read-only input
- Comment each DD with what the program expects
- Swap test and production by changing DSNs, never the program

QUICK SUMMARY

- DD name — what the program opens
- `DSN` + `DISP` — real dataset
- `SYSOUT=*` — job log
- `DD *` / `DUMMY` — in-stream / empty

The DISP Parameter

NEW / OLD / SHR / MOD · normal disposition · abnormal disposition · CATLG / DELETE / KEEP / PASS / UNCATLG · defaults · (NEW,CATLG,DELETE)

STATUS AND WHAT HAPPENS AFTER

DISP=(status,normal,abnormal) says whether the dataset exists and is shared, then what to do with it when the step ends **normally** and when it **abends**. The classic create pattern is (NEW,CATLG,DELETE): create it, keep and catalog it if the step works, delete it if the step fails.

KEY POINTS

- Status: NEW (create), OLD (exclusive), SHR (shared), MOD (append / create)
- Normal and abnormal default to "keep as it was" for existing datasets
- For NEW, the default is DELETE at step end
- `PASS` hands a dataset to a later step in the same job

DISP PATTERNS

```
//* Read, shared with other jobs
//IN       DD DSN=PROD.SALES,DISP=SHR
//* Update in place, nobody else may use it
//MASTER   DD DSN=PROD.ACCT.MASTER,DISP=OLD
//* Create; catalog if OK, delete on abend
//NEWOUT   DD DSN=USER.REPORT.DATA,
//            DISP=(NEW,CATLG,DELETE),
//            SPACE=(TRK,(10,5)),RECFM=FB,LRECL=133
//* Append to the end (creates it if it does not exist)
//LOG      DD DSN=USER.RUN.LOG,DISP=MOD
//* Delete an existing dataset
//OLDFILE  DD DSN=USER.OLD.DATA,DISP=(OLD,DELETE)
//* Keep on success, but keep for debugging on abend too
//WORK     DD DSN=USER.WORK.DATA,DISP=(NEW,CATLG,CATLG),
//            SPACE=(CYL,(1,1))
```

CODE EXPLAINED

→ SHR lets other jobs read the same dataset at the same time

→ OLD enqueues the dataset so only this job can use it

→ MOD positions writes after the last record

→ `(NEW,CATLG,CATLG)` keeps output from a failed run for inspection

MODERN EQUIVALENT

↔ **Python** `open(path, "r")` / `"w"` / `"a"`; `os.remove` on failure in `except`

↔ **Java** `StandardOpenOption.READ` / `CREATE_NEW` / `APPEND`; cleanup in `finally`

↔ **JavaScript** `fs.open(path, "r" | "wx" | "a")`; delete in a `catch` block

ABEND CHECK

✖ `IEF287I … NOT CATLGD 2`: a dataset with that name is already cataloged

✖ Job waits with "DATASET RESERVATION UNSUCCESSFUL": another job holds it with DISP=OLD

STATUS

VALUE	MEANING
NEW	Create now
OLD	Exists, exclusive
SHR	Exists, shared read
MOD	Append (or create)

DISPOSITIONS

VALUE	AT STEP END
CATLG	Keep + add to catalog
KEEP	Keep, catalog unchanged
DELETE	Delete (+ uncatalog)
PASS	Hand to later step
UNCATLG	Keep, remove from catalog

DEFAULTS

YOU CODE	MEANS
DISP=SHR	(SHR,KEEP,KEEP)
DISP=NEW	(NEW,DELETE,DELETE)
DISP=(NEW,CATLG)	abnormal = CATLG
no DISP	(NEW,DELETE,DELETE)

⚠ COMMON MISTAKES

- DISP=NEW alone, so the dataset is deleted when the step ends
- DISP=OLD on read-only input, blocking other jobs
- Creating a dataset whose name is already cataloged (NOT CATLGD 2)

✓ TIPS & BEST PRACTICES

- Default to `(NEW,CATLG,DELETE)` for outputs
- Delete-then-create in a prior IEFBR14 step makes jobs rerunnable
- Use SHR for anything you only read

QUICK SUMMARY

- `SHR` / `OLD` — read / exclusive
- `NEW` — create
- `(NEW,CATLG,DELETE)` — standard
- `MOD` — append

SPACE, UNIT & Volumes

SPACE=(TRK,(10,5)) · CYL / TRK / block units · directory blocks · RLSE · UNIT=SYSDA · VOL=SER= · AVGREC · SMS classes

ASKING FOR DISK SPACE

New datasets need **space**: a primary amount, a secondary amount added when it fills (up to 15 more extents per volume), and for a PDS, directory blocks. Space is given in tracks, cylinders, blocks or records. Most sites use **SMS**, which picks the volume, so `UNIT` and `VOL` are rarely needed.

KEY POINTS

- 1 cylinder = 15 tracks; 1 track ≈ 56 KB on 3390 disks
- `RLSE` gives back unused space when the dataset closes
- `AVGREC=K` with record units: `SPACE=(200,(50,10)),AVGREC=K` = 50,000 records of 200 bytes
- SMS: `STORCLAS`, `MGMTCLAS`, `DATACLAS` replace manual placement

SPACE FORMS

```
//* Tracks: 10 primary, 5 secondary, release unused
//OUT1     DD DSN=USER.SMALL,DISP=(NEW,CATLG,DELETE),
//            SPACE=(TRK,(10,5),RLSE),
//            RECFM=FB,LRECL=80
//* Cylinders for big files
//OUT2     DD DSN=USER.BIG,DISP=(NEW,CATLG,DELETE),
//            SPACE=(CYL,(100,50),RLSE),
//            RECFM=VB,LRECL=1004
//* PDS with 20 directory blocks
//LIB      DD DSN=USER.LOADLIB,DISP=(NEW,CATLG,DELETE),
//            SPACE=(CYL,(5,5,20)),RECFM=U,BLKSIZE=32760
//* By record count: 50K records of 200 bytes
//OUT3     DD DSN=USER.RECS,DISP=(NEW,CATLG,DELETE),
//            SPACE=(200,(50,10)),AVGREC=K,
//            RECFM=FB,LRECL=200
//* Explicit device and volume (non-SMS)
//OUT4     DD DSN=USER.PINNED,DISP=(NEW,CATLG,DELETE),
//            UNIT=SYSDA,VOL=SER=WORK01,SPACE=(TRK,1)
```

CODE EXPLAINED

→ Secondary space is added in extents as the dataset grows

→ The third number in SPACE is PDS directory blocks

→ `AVGREC=K` multiplies the counts by 1,024

→ `UNIT=SYSDA` is the generic disk device name

MODERN EQUIVALENT

↔ **Python** Disk quotas and volumes are handled by the OS; files grow freely

↔ **Java** Same: the JVM writes, the file system allocates

↔ **JavaScript** Same: Node writes, the file system allocates

ABEND CHECK

✖ `SB37`: no secondary space left or 16 extents used; increase SPACE

✖ `SD37`: no secondary amount coded; `SE37`: PDS directory or volume full

UNITS

UNIT	SIZE (3390)
TRK	~56 KB
CYL	15 tracks, ~850 KB
blksize	Blocks of that size
avg reclen + AVGREC	Records (U, K, M)

OUT-OF-SPACE ABENDS

CODE	CAUSE
SB37	Ran out of extents / space
SD37	Primary full, no secondary
SE37	Volume or directory full
SB14-0C	PDS directory full (on STOW)

⚠ COMMON MISTAKES

- No secondary quantity, so the job abends SD37 at the primary limit
- Tiny PDS directory, so adding members fails
- Asking for huge primary space that no volume can satisfy (job fails to allocate)

✓ TIPS & BEST PRACTICES

- Estimate: records × LRECL ÷ 56,000 ≈ tracks
- Always code RLSE on outputs
- Use PDSE to avoid directory-block limits

QUICK SUMMARY

- `SPACE=(unit,(pri,sec,dir))`
- `RLSE` — release unused
- `AVGREC` — size by records
- SB37 / SD37 / SE37 — out of space

DCB Attributes

RECFM=FB / VB / F / U · LRECL · BLKSIZE=0 (system-determined) · DSORG=PS / PO · LIKE= · DSNTYPE=LIBRARY

THE SHAPE OF THE RECORDS

DCB attributes describe the records in a dataset: their **format** (fixed or variable, blocked or not), their **length** and how many are grouped into a **block**. For new datasets you set them on the DD; existing datasets keep theirs in the catalog/label, and they must match what the program expects.

KEY POINTS

- Code them directly (`RECFM=FB`) or in `DCB=(…)`
- VB LRECL includes the 4-byte record descriptor word
- `BLKSIZE=0` lets the system pick an efficient block size
- `LIKE=` copies attributes from an existing dataset

IN COBOL

- RECFM=FB, LRECL=150 must match an FD with a 150-byte 01 record
- Variable files: LRECL = largest COBOL record + 4
- A mismatch gives FILE STATUS 39 on OPEN (or an S013 abend)

ATTRIBUTES FOR COMMON FILES

```
//* Fixed 80-byte card-image records (JCL, source)
//SRC      DD DSN=USER.COBOL.SRC,DISP=(NEW,CATLG,DELETE),
//            SPACE=(TRK,(20,10)),DSNTYPE=LIBRARY,
//            RECFM=FB,LRECL=80,BLKSIZE=0
//* Report with ASA control character
//RPT      DD DSN=USER.REPORT,DISP=(NEW,CATLG,DELETE),
//            SPACE=(TRK,(10,10)),RECFM=FBA,LRECL=133
//* Variable records up to 500 bytes of data
//VAR      DD DSN=USER.VARDATA,DISP=(NEW,CATLG,DELETE),
//            SPACE=(CYL,(1,1)),RECFM=VB,LRECL=504
//* Same attributes as an existing dataset
//COPY     DD DSN=USER.CUST.COPY,DISP=(NEW,CATLG,DELETE),
//            LIKE=PROD.CUSTOMER.MASTER
//* Old style
//OLDSTYLE DD DSN=USER.OLD,DISP=(NEW,CATLG,DELETE),
//            SPACE=(TRK,1),DCB=(RECFM=FB,LRECL=100)
```

CODE EXPLAINED

- → Source and JCL libraries are FB 80
- → FBA reports: byte 1 is the carriage control character
- → VB 504 = 500 bytes of data + 4-byte RDW
- → `LIKE=` copies RECFM, LRECL and space from the model

MODERN EQUIVALENT

- ↔ **Python** Text files with newlines; `struct` for fixed binary records
- ↔ **Java** `DataInputStream.readFully(buf)` for fixed-length records
- ↔ **JavaScript** `fs.read` into a `Buffer` of the record length

ABEND CHECK

- ✖ FILE STATUS 39 on OPEN: RECFM or LRECL do not match the program
- ✖ `IEC141I 013-20`: BLKSIZE not a multiple of LRECL for fixed records

RECFM

VALUE	RECORDS
F	Fixed, one per block
FB	Fixed, blocked
V	Variable, one per block
VB	Variable, blocked
FBA / VBA	With ASA control char
U	Undefined (load modules)

TYPICAL VALUES

DATASET	DCB
JCL / source	FB, 80
Report	FBA, 133
Load library	U, BLKSIZE 32760
Copybooks	FB, 80
Data files	FB/VB per layout

⚠ COMMON MISTAKES

- LRECL that does not match the COBOL record length
- Forgetting the 4 extra bytes on VB LRECL
- Hard-coding an old BLKSIZE that is no longer efficient

✓ TIPS & BEST PRACTICES

- Code BLKSIZE=0 on new datasets
- Check attributes in ISPF 3.4 with the `I` (info) line command
- Keep LRECL in the copybook header comment

QUICK SUMMARY

- `RECFM` — record format
- `LRECL` — record length
- `BLKSIZE=0` — system choice
- `LIKE=` — copy attributes

Special DD Names

DD NAMES WITH BUILT-IN MEANINGS

Some DD names are reserved or used by convention. **STEPLIB** and **JOBLIB** tell z/OS where to find programs. **SYSIN** carries control statements, **SYSPRINT** and **SYSOUT** collect messages, and the dump DDs capture diagnostics when a step abends.

KEY POINTS

- JOBLIB goes right after the JOB statement; STEPLIB after an EXEC
- STEPLIB overrides JOBLIB for its step
- Utilities read SYSIN and write SYSPRINT
- Most COBOL DISPLAY output lands in SYSOUT

IN COBOL

- COBOL `DISPLAY` → SYSOUT; `ACCEPT` → SYSIN
- Language Environment writes CEEDUMP when a COBOL step abends
- The compile step reads copybooks from SYSLIB

SPECIAL DDS IN ONE JOB

```
//USER01R  JOB (ACCT),'SPECIAL DDS',CLASS=A,MSGCLASS=X
//JOBLIB   DD DSN=USER.LOADLIB,DISP=SHR
//         DD DSN=PROD.LOADLIB,DISP=SHR
//*
//STEP1    EXEC PGM=PAYCALC
//STEPLIB  DD DSN=USER.TEST.LOADLIB,DISP=SHR
//SYSIN    DD *
RUNDATE=20261112
/*
//SYSOUT   DD SYSOUT=*
//CEEDUMP  DD SYSOUT=*
//SYSUDUMP DD SYSOUT=*
//*
//STEP2    EXEC PGM=IKJEFT01
//SYSTSPRT DD SYSOUT=*
//SYSTSIN  DD *
LISTCAT LEVEL(USER)
/*
```

CODE EXPLAINED

- → JOBLIB is searched for every step (two libraries concatenated)
- → STEP1 looks in USER.TEST.LOADLIB first because of STEPLIB
- → CEEDUMP and SYSUDUMP catch diagnostics if PAYCALC abends
- → IKJEFT01 runs TSO commands from SYSTSIN

MODERN EQUIVALENT

- ↔ **Python** `PYTHONPATH` (≈ STEPLIB), stdin (≈ SYSIN), stdout/stderr (≈ SYSOUT)
- ↔ **Java** `-classpath` (≈ STEPLIB), `System.in`, `System.out`
- ↔ **JavaScript** `NODE_PATH` / `node_modules` (≈ STEPLIB), `process.stdin`, `process.stdout`

ABEND CHECK

- ✖ S806 with STEPLIB coded: the library is fine but the member is in a different one
- ✖ No dump after an abend: add `CEEDUMP DD SYSOUT=*` (or SYSUDUMP)

SPECIAL DD NAMES

DD	PURPOSE
JOBLIB	Program library, whole job
STEPLIB	Program library, one step
SYSIN	Control statements / ACCEPT
SYSOUT	DISPLAY output, sort messages
SYSPRINT	Utility messages
SYSUDUMP / SYSABEND	System dumps
CEEDUMP	LE dump (COBOL)
SYSTSIN / SYSTSPRT	TSO batch in / out
SYSLIB	Copybooks / link libraries

⚠ COMMON MISTAKES

- JOBLIB placed after the first EXEC (it must come first)
- Missing SYSOUT DD, so DISPLAY output ends up somewhere unexpected
- Production STEPLIB left in a test job

✓ TIPS & BEST PRACTICES

- Concatenate test before production libraries in STEPLIB while testing
- Always include CEEDUMP for COBOL steps
- Code SYSPRINT for every utility step: that is where its errors go

QUICK SUMMARY

- `STEPLIB` / `JOBLIB` — find programs
- `SYSIN` — control input
- `SYSOUT` / `SYSPRINT` — messages
- `CEEDUMP` — COBOL abend dump

Sheet 13

Concatenation, Temp Datasets & Backward References

concatenated DDs · &&TEMP datasets · DISP=(NEW,PASS) · DSN=*.STEP1.DDNAME referbacks · VOL=REF · DCB=*.

CHAINING DATA THROUGH A JOB

Several datasets under one DD name are **concatenated** and read as one file. **Temporary datasets** (&&name) exist only for the life of the job and pass data between steps. **Backward references** (*.step.ddname) reuse a dataset or attributes defined earlier, so names are typed once.

KEY POINTS

- Concatenation: following DDs have no name
- Temp datasets are deleted when the job ends
- `PASS` keeps a dataset available to later steps
- Referbacks name the step (and proc step) plus DD

IN COBOL

- A COBOL program reading a concatenated DD sees one long file and one end-of-file
- Pass data from a sort step to a COBOL step through `&&SORTED` with no cleanup
- Concatenated datasets should share RECFM and LRECL

TEMP DATA BETWEEN STEPS

```
//STEP1    EXEC PGM=SORT
//SYSOUT   DD SYSOUT=*
//* Three months read as one input
//SORTIN   DD DSN=PROD.SALES.SEP,DISP=SHR
//         DD DSN=PROD.SALES.OCT,DISP=SHR
//         DD DSN=PROD.SALES.NOV,DISP=SHR
//SORTOUT  DD DSN=&&SORTED,DISP=(NEW,PASS),
//            SPACE=(CYL,(10,5)),RECFM=FB,LRECL=100
//SYSIN    DD *
  SORT FIELDS=(1,10,CH,A)
/*
//STEP2    EXEC PGM=SALESRPT
//SALESIN  DD DSN=&&SORTED,DISP=(OLD,DELETE)
//* Same data via a referback
//SALESIN2 DD DSN=*.STEP1.SORTOUT,DISP=(OLD,DELETE)
//RPTOUT   DD SYSOUT=*
```

CODE EXPLAINED

- → SORTIN reads September, October and November as one file
- → `&&SORTED` is created in STEP1 and passed
- → STEP2 reads it and deletes it when done
- → `*.STEP1.SORTOUT` refers back to the same dataset

MODERN EQUIVALENT

- ↔ **Python** `itertools.chain(open(a), open(b))`; `tempfile.NamedTemporaryFile`
- ↔ **Java** `SequenceInputStream`; `Files.createTempFile`
- ↔ **JavaScript** Stream piping (`stream.pipeline`); `fs.mkdtemp` + cleanup

ABEND CHECK

- ✖ `IEF212I … DATA SET NOT FOUND` on a referback: misspelled step or DD name
- ✖ Concatenation with different LRECLs: `IEC141I 013-xx` or truncated records

TEMP & REFERBACKS

FORM	MEANING
DSN=&&NAME	Temp dataset
DISP=(NEW,PASS)	Create, pass on
DISP=(OLD,DELETE)	Use, then delete
DSN=*.STEP.DD	Same dataset as earlier DD
DSN=*.STEP.PSTEP.DD	Through a proc step
DCB=*.STEP.DD	Copy DCB attributes
VOL=REF=*.STEP.DD	Same volume

⚠ COMMON MISTAKES

- Naming the concatenated DDs (only the first has a name)
- Forgetting PASS, so the temp dataset is deleted before the next step
- Single `&` instead of `&&` (that is a symbolic parameter)

✓ TIPS & BEST PRACTICES

- Use temp datasets for all intermediate files: no cleanup jobs needed
- Put the largest block size dataset first in a concatenation (older rules)
- Prefer referbacks over retyping long names

QUICK SUMMARY

- unnamed DDs — concatenation
- `&&TEMP` — job-life dataset
- `PASS` — keep for later steps
- `*.STEP.DD` — referback

Return Codes & COND

RC 0 / 4 / 8 / 12 / 16 · COND=(4,LT) reads backwards · COND=EVEN / ONLY · step vs job COND · MAXCC · flushed steps

SKIPPING STEPS ON ERRORS

Each step ends with a **return code** (condition code). The old COND parameter tells z/OS when to **skip** a step: COND=(4,LT) means "skip this step if 4 is less than any earlier RC". It reads backwards, which is why most new JCL uses IF/THEN/ELSE (sheet 15).

KEY POINTS

- RC convention: 0 OK, 4 warning, 8 error, 12 severe, 16 fatal
- COND tests are true → step is **bypassed**
- An abend normally bypasses later steps unless EVEN or ONLY
- COND on the JOB statement applies to every step

IN COBOL

- `MOVE 8 TO RETURN-CODE` in COBOL becomes the step's RC 8
- An abended COBOL step has no RC; COND=EVEN steps still run
- Utilities set RCs too: SORT 16 = failure, IDCAMS 8 = errors

COND EXAMPLES

```
//STEP010  EXEC PGM=VALIDATE
//* Skip if 4 < any previous RC  (run only if all RCs <= 4)
//STEP020  EXEC PGM=UPDATE,COND=(4,LT)
//* Skip if any previous RC = 0  (run only after an error)
//STEP030  EXEC PGM=ERRMAIL,COND=(0,EQ)
//* Test one specific step
//STEP040  EXEC PGM=REPORT,COND=(8,LE,STEP020)
//* Always run, even after an abend (cleanup)
//STEP050  EXEC PGM=CLEANUP,COND=EVEN
//* Run ONLY if an earlier step abended
//STEP060  EXEC PGM=ALERT,COND=ONLY
```

CODE EXPLAINED

- → Read `(4,LT)` as "if 4 is less than the RC, skip"
- → STEP020 runs when VALIDATE ends with RC 0 or 4
- → STEP040 is skipped if 8 ≤ STEP020's RC
- → EVEN runs after abends; ONLY runs only after abends

MODERN EQUIVALENT

- ↔ **Python** `sys.exit(4)`; a wrapper checks `result.returncode <= 4` before the next step
- ↔ **Java** `System.exit(4)`; the scheduler checks the exit code
- ↔ **JavaScript** `process.exitCode = 4`; `step1 && step2` in npm scripts skips on non-zero

ABEND CHECK

- ✖ `IEF202I STEP020 - STEP WAS NOT RUN BECAUSE OF COND`: the COND test was true
- ✖ `FLUSH` in the step summary: bypassed because an earlier step abended

OPERATORS

CODE	SKIP WHEN
GT	value > RC
GE	value ≥ RC
EQ	value = RC
LT	value < RC
LE	value ≤ RC
NE	value ≠ RC

COMMON PATTERNS

COND	RUNS WHEN
(4,LT)	All RCs ≤ 4
(0,NE)	All RCs = 0
(8,LE)	All RCs < 8
(0,EQ)	No earlier RC = 0
EVEN	Always, even after abend
ONLY	Only after an abend

⚠ COMMON MISTAKES

- Reading COND forwards and skipping exactly the wrong steps
- COND=(0,NE) on a step after a utility that returns 4 for warnings
- Expecting COND to catch abends without EVEN or ONLY

✓ TIPS & BEST PRACTICES

- Translate every COND into words in a comment
- Prefer IF/THEN/ELSE in new JCL
- Make programs set RETURN-CODE deliberately (0, 4, 8, 16)

QUICK SUMMARY

- RC 0 / 4 / 8 / 16
- `COND` true → step skipped
- `(4,LT)` — run if RCs ≤ 4
- `EVEN` / `ONLY` — abend handling

IF / THEN / ELSE & Restart

READABLE CONDITIONAL STEPS

IF/THEN/ELSE/ENDIF statements wrap steps in conditions that read forwards: IF (RC <= 4) THEN runs the steps when the condition is true. You can test the highest RC so far, a specific step's RC, abends and abend codes. RESTART= reruns a failed job from a chosen step.

KEY POINTS

- `RC` alone = highest RC of all earlier steps
- `STEP1.RC` tests one step; `STEP1.PSTEP.RC` a proc step
- `ABEND` is true if any earlier step abended
- IF blocks can nest up to 15 levels

IF BLOCKS AND RESTART

```
//STEP010  EXEC PGM=VALIDATE
//         IF (STEP010.RC <= 4) THEN
//STEP020  EXEC PGM=UPDATE
//         IF (STEP020.RC = 0) THEN
//STEP030  EXEC PGM=REPORT
//         ENDIF
//         ELSE
//STEP040  EXEC PGM=ERRMAIL,PARM='VALIDATION FAILED'
//         ENDIF
//*
//         IF (ABEND | STEP020.ABENDCC = S0C7) THEN
//STEP090  EXEC PGM=ALERT
//         ENDIF
//*
//* Rerun from STEP020 after fixing the data:
//* //USER01P JOB (ACCT),'RERUN',CLASS=A,RESTART=STEP020
```

CODE EXPLAINED

→ STEP020 runs only if VALIDATE ended with RC 0–4

→ REPORT runs only if UPDATE returned 0

→ STEP040 runs when validation failed

→ `RESTART=STEP020` skips STEP010 on the rerun

MODERN EQUIVALENT

↔ **Python** `if result.returncode <= 4: run("update")` in a Python driver

↔ **Java** `if (exitCode <= 4) { … }` in a launcher, or Spring Batch flow decisions

↔ **JavaScript** GitHub Actions `if: steps.validate.outcome == 'success'`

ABEND CHECK

✖ `JCL ERROR` on an IF block: an IF without a matching ENDIF (or the reverse)

✖ Restart fails because STEP010 created a temp dataset STEP020 needs: restart earlier or catalog it

EXPRESSIONS

TERM	MEANING
RC	Highest RC so far
step.RC	One step's RC
ABEND / ¬ABEND	Any abend / none
step.ABEND	That step abended
step.ABENDCC=S0C7	Specific abend
step.RUN	That step ran
& | ¬ (AND OR NOT)	Combine

RESTART

FORM	EFFECT
RESTART=STEP020	Start at that step
RESTART=PROC.STEP	Start in a proc step
TYPRUN=HOLD	Submit held, release later
GDG (+1) on restart	Watch generation numbers

⚠ COMMON MISTAKES

- Comparing `RC > 4` when you meant `<=` (logic inverted from COND habits)
- Forgetting ENDIF
- Restarting at a step that depends on temp datasets from skipped steps

✓ TIPS & BEST PRACTICES

- Use IF/THEN/ELSE for all new conditional logic
- Design steps to be rerunnable: delete outputs first, then create
- Comment each IF with the business reason

QUICK SUMMARY

- `IF (cond) THEN … ENDIF`
- `RC` / `step.RC` — return codes
- `ABEND` / `ABENDCC` — failures
- `RESTART=` — rerun from a step

In-Stream & Cataloged Procedures

REUSABLE JCL

A **procedure** (proc) is a named set of steps you call with one EXEC. **In-stream** procs sit inside the job between PROC and PEND; **cataloged** procs live as members in a procedure library found through JCLLIB or the system proclib. Callers override or add DDs with `procstep.ddname`.

KEY POINTS

- A proc cannot contain JOB statements or `DD *` (in older z/OS)
- IBM supplies procs like IGYWCL (compile + link)
- `JCLLIB ORDER=` lists your proc libraries, searched first
- Overrides must follow the EXEC in proc-step order

DEFINE AND CALL A PROC

```
//USER01P  JOB (ACCT),'PROCS',CLASS=A,MSGCLASS=X
//         JCLLIB ORDER=(USER.PROCLIB)
//*
//* In-stream procedure
//RUNPGM   PROC PGM=,INDSN=
//STEP1    EXEC PGM=&PGM
//STEPLIB  DD DSN=USER.LOADLIB,DISP=SHR
//INFILE   DD DSN=&INDSN,DISP=SHR
//SYSOUT   DD SYSOUT=*
//         PEND
//*
//RUN1     EXEC RUNPGM,PGM=PAYCALC,INDSN=PROD.PAY.TRANS
//*
//* Cataloged proc USER.PROCLIB(NIGHTLY) with overrides
//RUN2     EXEC NIGHTLY
//STEP1.INFILE DD DSN=USER.TEST.TRANS,DISP=SHR
//STEP1.EXTRA  DD DSN=USER.EXTRA.DATA,DISP=SHR
```

CODE EXPLAINED

- → RUNPGM takes two symbolic parameters, PGM and INDSN
- → RUN1 calls it, filling in both values
- → `STEP1.INFILE` replaces the INFILE DD in the NIGHTLY proc
- → `STEP1.EXTRA` adds a DD the proc did not have

MODERN EQUIVALENT

- ↔ **Python** A shell function or Python function wrapping a command with arguments
- ↔ **Java** A reusable method or Spring Batch step bean
- ↔ **JavaScript** A reusable GitHub Actions workflow or npm script with arguments

ABEND CHECK

- ✖ `IEFC612I PROCEDURE NOT FOUND`: missing JCLLIB library or misspelled proc name
- ✖ Override ignored: the procstep or DD name is wrong, or overrides are out of order

PROC STATEMENTS

STATEMENT	PURPOSE
`//name PROC parms`	Start proc, set defaults
`// PEND`	End in-stream proc
`// JCLLIB ORDER=(lib)`	Where to find procs
`//step EXEC procname`	Call proc
`//pstep.dd DD …`	Override / add DD
`PARM.pstep=`	Override EXEC PARM

⚠ COMMON MISTAKES

- Overrides in a different order than the proc steps
- Overriding a DD name that does not exist in the proc (it is added instead)
- Editing a shared production proc to fix one job

✓ TIPS & BEST PRACTICES

- Keep site procs read-only; override in your JCL
- Expand procs in the job log with MSGLEVEL=(1,1) to see the final JCL
- Name proc steps clearly so overrides read well

QUICK SUMMARY

- `PROC` / `PEND` — in-stream proc
- `JCLLIB` — proc libraries
- `EXEC procname` — call
- `pstep.dd` — override

Symbolic Parameters & SET

VARIABLES IN JCL

Symbolic parameters (&NAME) are placeholders replaced before the job runs. Defaults are set on the PROC statement or with SET, and callers override them on EXEC. **System symbols** such as &SYSUID and dates are filled in by z/OS. A period ends a symbol, so &HLQ..DATA becomes USER.DATA.

KEY POINTS

- Symbol names: 1–7 characters after `&`
- A null symbol (`PARM=`) removes the text
- Two periods are needed when a period follows a symbol
- `&&` is a temp dataset, not a symbol

SYMBOLS AND SET

```
//USER01S  JOB (ACCT),'SYMBOLS',CLASS=A,MSGCLASS=X
//         SET HLQ=USER,ENV=TEST,RUNDT=20261112
//*
//STEP1    EXEC PGM=PAYCALC,PARM='&RUNDT'
//INFILE   DD DSN=&HLQ..&ENV..PAY.TRANS,DISP=SHR
//*        resolves to USER.TEST.PAY.TRANS
//OUTFILE  DD DSN=&HLQ..PAY.D&LYYMMDD,
//            DISP=(NEW,CATLG,DELETE),SPACE=(TRK,(5,5))
//*        e.g. USER.PAY.D261112
//*
//* Symbols inside in-stream data (z/OS 2.1+)
//         EXPORT SYMLIST=(RUNDT)
//STEP2    EXEC PGM=RPT
//SYSIN    DD *,SYMBOLS=JCLONLY
RUN DATE &RUNDT
/*
```

CODE EXPLAINED

→ SET defines symbols for the rest of the job

→ `&HLQ..&ENV..PAY` keeps a dot between each part

→ `&LYYMMDD` is the local date when the job is read in

→ `SYMBOLS=JCLONLY` replaces exported symbols inside SYSIN

MODERN EQUIVALENT

↔ **Python** f-strings / env vars: `f"{hlq}.{env}.PAY.TRANS"`

↔ **Java** `String.format("%s.%s.PAY.TRANS", hlq, env)` or properties

↔ **JavaScript** Template literals: `${hlq}.${env}.PAY.TRANS`

ABEND CHECK

✖ `IEFC657I THE SYMBOL … WAS NOT USED` / unresolved `&X` in a DSN: misspelled or never set

✖ Name like `USERPAY` instead of `USER.PAY` : only one period after the symbol

SYSTEM SYMBOLS

SYMBOL	VALUE
&SYSUID	Submitter user ID
&LYYMMDD	Local date YYMMDD
&LYR4 / &LMON / &LDAY	Year, month, day
&LHHMMSS	Local time
&SYSNAME	System name
&JOBNAME	Job name (z/OS 2.x)

RULES

RULE	EXAMPLE
End with a period	&HLQ..DATA
Default on PROC	//P PROC ENV=TEST
Override on EXEC	//S EXEC P,ENV=PROD
Null value	PARM= removes it
Quoted value	TEXT='A B'

⚠ COMMON MISTAKES

- One period after a symbol before a qualifier, which joins them
- Expecting symbols to be replaced in in-stream data without SYMBOLS=
- Using symbols for things that should be program parameters

✓ TIPS & BEST PRACTICES

- Put environment-specific values (HLQ, ENV) in one SET at the top
- Use `&SYSUID` in test JCL so anyone can run it
- Check the expanded JCL in the job log (MSGLEVEL=(1,1))

QUICK SUMMARY

- `&NAME` — symbolic parameter
- `SET` — define symbols
- `..` — symbol then period
- `&SYSUID` / `&LYYMMDD` — system symbols

INCLUDE & Overrides

INCLUDE MEMBER= · overriding EXEC params PARM.STEP1= · nullifying parameters · override order · common override mistakes

PULLING IN JCL AND CHANGING PROCS

INCLUDE MEMBER= pastes a JCL fragment from a library in the JCLLIB list, like a copybook for JCL. When calling a proc, PARM.procstep= and other keyword.procstep= values override EXEC parameters, and DD overrides can change, add or nullify individual parameters.

KEY POINTS

- INCLUDE members hold JCL statements, not a whole job
- EXEC overrides: `PARM.STEP1=`, `REGION.STEP1=`, `TIME.STEP1=`
- A DD override changes only the parameters you code
- Code a parameter with no value to nullify it: `UNIT=`

INCLUDE AND OVERRIDES

```
//USER01I  JOB (ACCT),'INCLUDE',CLASS=A,MSGCLASS=X
//         JCLLIB ORDER=(USER.JCLLIB,USER.PROCLIB)
//*
//STEP1    EXEC PGM=PAYCALC
//         INCLUDE MEMBER=STDLIBS      STEPLIB + SYSOUT DDs
//INFILE   DD DSN=PROD.PAY.TRANS,DISP=SHR
//*
//* Override EXEC parameters inside the proc NIGHTLY
//RUN      EXEC NIGHTLY,
//         PARM.STEP1='2026-11',
//         REGION.STEP2=256M
//* Change only DISP and SPACE on an existing DD
//STEP2.OUTFILE DD DISP=(NEW,CATLG,CATLG),
//            SPACE=(CYL,(20,10))
//* Nullify the proc's UNIT and VOL on this DD
//STEP2.WORK   DD UNIT=,VOL=
//* Send a DD to a dummy for this run
//STEP3.AUDIT  DD DUMMY
```

CODE EXPLAINED

→ STDLIBS might hold STEPLIB, SYSOUT and CEEDUMP DDs used everywhere

→ `PARM.STEP1=` overrides the PARM on proc step STEP1

→ OUTFILE keeps its DSN; only DISP and SPACE change

→ `UNIT=,VOL=` removes those parameters so SMS chooses

MODERN EQUIVALENT

↔ **Python** `import` of shared config; keyword overrides on a function call

↔ **Java** Shared config classes; overriding defaults with builder methods

↔ **JavaScript** Shared YAML anchors or composite actions; `with:` inputs override defaults

ABEND CHECK

✖ `IEFC612I PROCEDURE NOT FOUND` for INCLUDE members too: JCLLIB missing the library

✖ DD override has no effect: procstep name misspelled, so a new DD was added instead

OVERRIDE FORMS

FORM	EFFECT
`PARM.pstep='x'`	Replace PARM
`REGION.pstep=`	Replace REGION
`COND.pstep=`	Replace COND
`//pstep.dd DD parm=…`	Change DD params
`//pstep.dd DD parm=`	Nullify a param
`//pstep.dd DD DUMMY`	Make it dummy
`//pstep.newdd DD …`	Add a DD

⚠ COMMON MISTAKES

- Overrides listed out of proc-step order
- Coding a full DD when only one parameter needed to change
- INCLUDE members that contain JOB or PROC statements

✓ TIPS & BEST PRACTICES

- Collect standard DDs in an INCLUDE member
- List every override with a comment explaining why
- Check the expanded JCL to confirm each override applied

QUICK SUMMARY

- `INCLUDE MEMBER=` — paste JCL
- `PARM.pstep=` — EXEC override
- partial DD override
- `parm=` (empty) — nullify

IEFBR14 & IEBGENER

allocate with IEFBR14 · delete with DISP=(MOD,DELETE) · IEBGENER copy · SYSUT1 / SYSUT2 · SYSIN DD DUMMY · ICEGENER

THE TWO MOST-USED UTILITIES

IEFBR14 does nothing at all: it exists so DD statements can create or delete datasets. **IEBGENER** copies a sequential dataset (SYSUT1) to another (SYSUT2), printing or creating it on the way. Many sites alias IEBGENER to the faster **ICEGENER** from DFSORT.

KEY POINTS

- IEFBR14 always returns RC 0
- `DISP=(MOD,DELETE)` deletes whether or not the dataset exists
- IEBGENER with `SYSIN DD DUMMY` is a straight copy
- SYSUT2 can be SYSOUT=* to print a dataset

DELETE, CREATE, COPY

```
//* Delete if it exists (no error if it does not)
//DELETE   EXEC PGM=IEFBR14
//DD1      DD DSN=USER.PAY.COPY,DISP=(MOD,DELETE),
//            SPACE=(TRK,0)
//*
//* Copy a dataset, creating the output
//COPY     EXEC PGM=IEBGENER
//SYSPRINT DD SYSOUT=*
//SYSIN    DD DUMMY
//SYSUT1   DD DSN=PROD.PAY.TRANS,DISP=SHR
//SYSUT2   DD DSN=USER.PAY.COPY,DISP=(NEW,CATLG,DELETE),
//            SPACE=(CYL,(5,5),RLSE),LIKE=PROD.PAY.TRANS
//*
//* Print a dataset to the job log
//PRINT    EXEC PGM=IEBGENER
//SYSPRINT DD SYSOUT=*
//SYSIN    DD DUMMY
//SYSUT1   DD DSN=USER.PAY.COPY,DISP=SHR
//SYSUT2   DD SYSOUT=*
```

CODE EXPLAINED

- → MOD creates the dataset if missing, then DELETE removes it: no failure either way
- → This delete-then-create pattern makes the job rerunnable
- → `LIKE=` copies the input's attributes to the output
- → Printing to SYSOUT is a quick way to look at a file in the job log

MODERN EQUIVALENT

- ↔ **Python** `Path(p).unlink(missing_ok=True)` ; `shutil.copyfile(a, b)`
- ↔ **Java** `Files.deleteIfExists(p)` ; `Files.copy(a, b)`
- ↔ **JavaScript** `fs.rmSync(p, { force: true })` ; `fs.copyFileSync(a, b)`

ABEND CHECK

- ✖ IEBGENER RC 12: missing SYSUT1/SYSUT2 or bad DCB; read SYSPRINT
- ✖ `NOT CATLGD 2` on SYSUT2: the output name already exists; add a delete step first

IEFBR14 USES

DD CODING	EFFECT
DISP=(NEW,CATLG)	Create + catalog
DISP=(MOD,DELETE)	Delete if present
DISP=(OLD,DELETE)	Delete (must exist)
DISP=(OLD,UNCATLG)	Uncatalog only

IEBGENER DDS

DD	ROLE
SYSUT1	Input
SYSUT2	Output
SYSIN	Control (DUMMY = copy)
SYSPRINT	Messages

⚠ COMMON MISTAKES

- Creating an output that already exists because there was no delete step
- Forgetting SYSPRINT, so IEBGENER errors are lost
- Expecting IEFBR14 to do anything without DD statements

✓ TIPS & BEST PRACTICES

- Start rerunnable jobs with an IEFBR14 delete step
- Use `LIKE=` or copy DCB from the input
- Use IEBGENER + SYSOUT to peek at small datasets

QUICK SUMMARY

- IEFBR14 — DD-only step
- `(MOD,DELETE)` — safe delete
- IEBGENER — SYSUT1 → SYSUT2
- `SYSIN DD DUMMY` — plain copy

IEBCOPY

copy PDS / PDSE members · COPY OUTDD=… INDD=… · SELECT MEMBER= · EXCLUDE · compress in place · unload / reload

COPYING LIBRARIES AND MEMBERS

IEBCOPY copies whole partitioned datasets or selected members between libraries. It also **compresses** a PDS in place to reclaim space left by deleted and replaced members, and **unloads** a library to a sequential file for transfer or backup.

KEY POINTS

- INDD and OUTDD name DD statements, not datasets
- `SELECT` copies only listed members; `EXCLUDE` skips them
- `R` after a member name replaces an existing member
- PDSEs reuse space and do not need compressing

COPY MEMBERS AND COMPRESS

```
//COPYMEM  EXEC PGM=IEBCOPY
//SYSPRINT DD SYSOUT=*
//FROMLIB  DD DSN=PROD.COBOL.SRC,DISP=SHR
//TOLIB    DD DSN=USER.COBOL.SRC,DISP=SHR
//SYSIN    DD *
  COPY OUTDD=TOLIB,INDD=FROMLIB
  SELECT MEMBER=(PAYCALC,PAYRPT,(TAXCALC,TAXNEW,R))
/*
//*
//* Compress a PDS in place
//COMPRESS EXEC PGM=IEBCOPY
//SYSPRINT DD SYSOUT=*
//LIB      DD DSN=USER.OLD.PDS,DISP=OLD
//SYSIN    DD *
  COPY OUTDD=LIB,INDD=LIB
/*
//*
//* Unload a library to a sequential file
//UNLOAD   EXEC PGM=IEBCOPY
//SYSPRINT DD SYSOUT=*
//IN       DD DSN=USER.COBOL.SRC,DISP=SHR
//OUT      DD DSN=USER.COBOL.UNLOAD,DISP=(NEW,CATLG,DELETE),
//            SPACE=(CYL,(5,5))
//SYSIN    DD *
  COPY OUTDD=OUT,INDD=IN
/*
```

CODE EXPLAINED

- → Only three members are copied; TAXCALC is renamed TAXNEW and replaces an existing one
- → INDD = OUTDD on the same library compresses it
- → An unload makes a sequential file you can send by FTP
- → DISP=OLD prevents others updating the library during compress

MODERN EQUIVALENT

- ↔ **Python** `shutil.copytree(src, dst)` or copy chosen files
- ↔ **Java** `Files.walk` + `Files.copy` for selected files
- ↔ **JavaScript** `fs.cpSync(src, dst, { recursive: true, filter })`

ABEND CHECK

- ✖ SYSPRINT reports a member not found: a SELECT name does not exist in INDD
- ✖ Compressing a PDS that others are using: corruption risk; use DISP=OLD

CONTROL STATEMENTS

STATEMENT	DOES
`COPY OUTDD=o,INDD=i`	Copy all members
`INDD=(a,b)`	Several inputs
`SELECT MEMBER=(m1,m2)`	Only these
`SELECT MEMBER=((old,new))`	Rename
`(m,,R) / (old,new,R)`	Replace existing
`EXCLUDE MEMBER=(m)`	Skip these
`COPY OUTDD=x,INDD=x`	Compress

⚠ COMMON MISTAKES

- Forgetting `R`, so existing members are not replaced (and the copy is skipped)
- Using SELECT and EXCLUDE in the same COPY group
- Compressing a busy production library

✓ TIPS & BEST PRACTICES

- Back up a PDS (unload) before risky changes
- Move libraries to PDSE to stop needing compresses
- Read SYSPRINT: it lists every member copied or skipped

QUICK SUMMARY

- IEBCOPY — library copies
- `SELECT` / `EXCLUDE` — choose members
- `R` — replace
- INDD = OUTDD — compress

IDCAMS

DEFINE CLUSTER (KSDS) · KEYS / RECORDSIZE · REPRO · PRINT · DELETE … PURGE · LISTCAT · SET MAXCC=0

THE VSAM AND CATALOG UTILITY

IDCAMS (Access Method Services) defines, loads, prints and deletes **VSAM** datasets and manages catalog entries. It takes commands in SYSIN: `DEFINE CLUSTER` creates a KSDS, `REPRO` copies data in or out, `PRINT` dumps records and `LISTCAT` shows catalog details.

KEY POINTS

- Commands continue with a trailing `-`
- `IF LASTCC = 8 THEN SET MAXCC = 0` ignores "not found" on deletes
- KEYS(length offset) must match the program's record key
- REPRO can load a KSDS from a sorted sequential file

DEFINE, LOAD, PRINT

```
//VSAMDEF  EXEC PGM=IDCAMS
//SYSPRINT DD SYSOUT=*
//SEQIN    DD DSN=USER.ACCT.SORTED,DISP=SHR
//SYSIN    DD *
  DELETE USER.ACCT.KSDS CLUSTER PURGE
  IF LASTCC = 8 THEN SET MAXCC = 0

  DEFINE CLUSTER (NAME(USER.ACCT.KSDS) -
         INDEXED -
         KEYS(10 0) -
         RECORDSIZE(100 100) -
         FREESPACE(10 10) -
         CYLINDERS(5 2)) -
       DATA  (NAME(USER.ACCT.KSDS.DATA)) -
       INDEX (NAME(USER.ACCT.KSDS.INDEX))

  REPRO INFILE(SEQIN) OUTDATASET(USER.ACCT.KSDS)
  PRINT INDATASET(USER.ACCT.KSDS) CHARACTER COUNT(5)
  LISTCAT ENTRIES(USER.ACCT.KSDS) ALL
/*
```

CODE EXPLAINED

→ The delete makes the job rerunnable; RC 8 (not found) is reset to 0

→ `KEYS(10 0)` : 10-byte key starting at offset 0

→ `RECORDSIZE(100 100)` : average and maximum record length

→ REPRO loads the KSDS; input must be in key order

MODERN EQUIVALENT

↔ **Python** `CREATE TABLE … PRIMARY KEY` then a bulk `INSERT` (sqlite3)

↔ **Java** JDBC DDL + batch inserts, or a `TreeMap` persisted to disk

↔ **JavaScript** SQL DDL via a driver, or an embedded key-value store

ABEND CHECK

✖ IDCAMS RC 8 `IDC3012I ENTRY NOT FOUND` on DELETE: harmless with the LASTCC reset

✖ REPRO RC 12 `IDC3314I` duplicate / out-of-sequence key: sort the input by key first

COMMANDS

COMMAND	DOES
`DEFINE CLUSTER`	Create VSAM
`DEFINE GDG`	Create GDG base
`DEFINE AIX / PATH`	Alternate index
`REPRO`	Copy / load / unload
`PRINT`	Dump records
`DELETE`	Delete + uncatalog
`LISTCAT`	Catalog info
`ALTER`	Change attributes

CLUSTER TYPES

KEYWORD	VSAM TYPE
`INDEXED`	KSDS
`NONINDEXED`	ESDS
`NUMBERED`	RRDS
`LINEAR`	LDS

⚠ COMMON MISTAKES

- KEYS offset or length not matching the COBOL RECORD KEY (status 39)
- REPRO from unsorted input into a KSDS
- Missing `-` continuation, so the rest of the command is lost

✓ TIPS & BEST PRACTICES

- Keep VSAM DEFINE jobs in source control beside the programs
- Use FREESPACE on files with many inserts
- Use `PRINT … CHARACTER COUNT(n)` to check data quickly

QUICK SUMMARY

- IDCAMS — VSAM utility
- `DEFINE CLUSTER` — create KSDS
- `REPRO` — load / copy
- `SET MAXCC=0` — ignore not-found

DFSORT

SORT FIELDS=(1,10,CH,A) · CH / ZD / PD / BI formats · INCLUDE COND= / OMIT · INREC / OUTREC BUILD= · SUM FIELDS=NONE · ICETOOL

SORTING, FILTERING AND REFORMATTING

DFSORT (PGM=SORT) sorts, merges, filters and reformats records with control statements in SYSIN. Fields are given as **position, length, format, order**. INCLUDE keeps matching records, OUTREC BUILD rearranges fields, and SUM FIELDS=NONE removes duplicate keys.

KEY POINTS

- Positions start at 1; for VB data, position 5 is the first data byte
- Formats must match the data: CH text, ZD zoned, PD packed
- SORTIN in, SORTOUT out, SYSOUT messages
- RC 0 = success, 16 = failure

IN COBOL

- Packed `PIC S9(7)V99 COMP-3` = 5 bytes → format PD, length 5
- Zoned `PIC S9(5)` = 5 bytes → format ZD
- Positions come from the copybook: offset + 1

SORT, FILTER, REFORMAT

```
//SORTSTEP EXEC PGM=SORT
//SYSOUT   DD SYSOUT=*
//SORTIN   DD DSN=PROD.SALES.NOV,DISP=SHR
//SORTOUT  DD DSN=USER.SALES.IAMN,DISP=(NEW,CATLG,DELETE),
//            SPACE=(CYL,(5,5),RLSE)
//SYSIN    DD *
* Keep Iowa and Minnesota, sales over 1000.00
  INCLUDE COND=((21,2,CH,EQ,C'IA',OR,21,2,CH,EQ,C'MN'),
                AND,51,5,PD,GT,+100000)
* State ascending, sales descending
  SORT FIELDS=(21,2,CH,A,51,5,PD,D)
* Output: account, state, sales
  OUTREC BUILD=(1,10,21,2,51,5)
/*
//*
//DEDUP    EXEC PGM=SORT
//SYSOUT   DD SYSOUT=*
//SORTIN   DD DSN=USER.CUST.ALL,DISP=SHR
//SORTOUT  DD DSN=USER.CUST.UNIQUE,DISP=(NEW,CATLG,DELETE),
//            SPACE=(CYL,(5,5),RLSE)
//SYSIN    DD *
  SORT FIELDS=(1,10,CH,A)
  SUM FIELDS=NONE
/*
```

CODE EXPLAINED

- → Bytes 21–22 hold the state; bytes 51–55 a packed amount
- → Packed 100000 with 2 implied decimals = 1000.00
- → OUTREC keeps 17 bytes: account, state, amount
- → `SUM FIELDS=NONE` keeps one record per key

MODERN EQUIVALENT

- ↔ **Python** `sorted(filter(lambda r: r.state in ("IA","MN"), rows), key=…)`
- ↔ **Java** `rows.stream().filter(…).sorted(comparing(…)).map(…)`
- ↔ **JavaScript** `rows.filter(…).sort(…).map(r => ({acct, state, sales}))`

ABEND CHECK

- ✖ DFSORT RC 16 `ICE…A` message in SYSOUT: read the first A-level message
- ✖ `ICE043A` / wrong data in output: field position or format does not match the layout

FORMATS

CODE	DATA
CH	Characters
ZD	Zoned decimal
PD	Packed decimal
BI	Unsigned binary
FI	Signed binary
Y4T	Date CCYYMMDD

STATEMENTS

STATEMENT	DOES
SORT FIELDS=(p,l,f,A|D)	Sort
MERGE FIELDS=…	Merge sorted inputs
SORT FIELDS=COPY	Copy, no sort
INCLUDE / OMIT COND=	Filter
INREC / OUTREC BUILD=	Reformat
SUM FIELDS=(p,l,f)	Total by key
OUTFIL	Several outputs

⚠ COMMON MISTAKES

- Positions counted from 0 (DFSORT starts at 1)
- Treating packed data as CH, which sorts by raw bytes
- Forgetting the 4-byte RDW when coding positions on VB files

✓ TIPS & BEST PRACTICES

- Copy field positions from the COBOL compile listing MAP
- Use `SORT FIELDS=COPY` with INCLUDE for fast filtering
- Use OUTFIL to split one input into several outputs in one pass

QUICK SUMMARY

- `SORT FIELDS=(p,l,f,o)`
- `INCLUDE COND=` — filter
- `OUTREC BUILD=` — reformat
- `SUM FIELDS=NONE` — dedupe

TSO in Batch & Other Utilities

IKJEFT01 · SYSTSIN commands · REXX in batch · DSN RUN PROGRAM (DB2) · BPXBATCH for USS · IEBUPDTE

RUNNING COMMANDS IN A JOB

IKJEFT01 runs TSO commands, CLISTs and REXX execs in batch, reading them from SYSTSIN. It is also how batch **DB2** COBOL programs are run, through the DSN command. **BPXBATCH** runs UNIX (USS) shell commands, and **IEBUPDTE** creates or updates PDS members from in-stream data.

KEY POINTS

- SYSTSPRT gets the command output
- The step RC is the highest RC of the commands
- DB2 programs must run under DSN so they connect to the subsystem
- BPXBATCH writes STDOUT/STDERR to DDs or files

IN COBOL

- A DB2 COBOL program runs as `RUN PROGRAM(name) PLAN(name)` under DSN
- The program's file DDs (SELECT ASSIGN) go in the same step
- Its RETURN-CODE becomes the IKJEFT01 step RC

IKJEFT01, DB2 AND USS

```
//* TSO commands in batch
//TSOCMD   EXEC PGM=IKJEFT01
//SYSTSPRT DD SYSOUT=*
//SYSTSIN  DD *
  LISTCAT LEVEL(USER.PAY)
  DELETE 'USER.PAY.OLD'
/*
//*
//* Run a DB2 COBOL program
//DB2RUN   EXEC PGM=IKJEFT01,DYNAMNBR=20
//STEPLIB  DD DSN=DSNC10.SDSNLOAD,DISP=SHR
//         DD DSN=USER.LOADLIB,DISP=SHR
//SYSTSPRT DD SYSOUT=*
//SYSOUT   DD SYSOUT=*
//CUSTRPT  DD SYSOUT=*
//SYSTSIN  DD *
  DSN SYSTEM(DB2P)
  RUN PROGRAM(CUSTRPT) PLAN(CUSTPLAN)
  END
/*
//*
//* UNIX command
//USSCMD   EXEC PGM=BPXBATCH,
//         PARM='SH ls -l /u/user01/data'
//STDOUT   DD SYSOUT=*
//STDERR   DD SYSOUT=*
```

CODE EXPLAINED

- TSOCMD lists and deletes datasets from a job
- DSN SYSTEM(DB2P) connects to the DB2 subsystem named DB2P
- CUSTRPT is a DD the COBOL program writes to
- BPXBATCH runs `ls` in the z/OS UNIX shell

MODERN EQUIVALENT

- **Python** `subprocess.run(["ls", "-l"])` from a Python job
- **Java** `ProcessBuilder("ls", "-l")`
- **JavaScript** `child_process.execSync("ls -l")`

ABEND CHECK

- `DSNE… SUBSYSTEM NOT DEFINED` / not connected: wrong `SYSTEM()` name or missing SDSNLOAD in STEPLIB
- SQLCODE -805 / -818: the plan or package does not match the program; rebind

UTILITIES

PROGRAM	USE
IKJEFT01	TSO / REXX / DB2 in batch
BPXBATCH	USS shell commands
IEBUPDTE	Create/update PDS members
IEHLIST	List VTOC / PDS directory
ADRDSSU	Dump / restore (DFSMSdss)
IEBCOMPR	Compare datasets

DSN SUBCOMMANDS

COMMAND	DOES
DSN SYSTEM(x)	Connect to DB2
RUN PROGRAM(p) PLAN(q)	Run COBOL + SQL
BIND PACKAGE(…)	Bind DBRM
END	Disconnect

⚠ COMMON MISTAKES

- Running a DB2 program with `EXEC PGM=` directly (no DB2 connection)
- No SYSTSPRT DD, so TSO error messages are lost
- Unquoted dataset names in TSO, which get your prefix added

✓ TIPS & BEST PRACTICES

- Check SYSTSPRT first when an IKJEFT01 step fails
- Keep DB2 run JCL in a proc with SYSTEM and PLAN as symbols
- Use BPXBATCH for Java or shell steps in the same job

QUICK SUMMARY

- IKJEFT01 — TSO in batch
- `DSN … RUN PROGRAM` — DB2 COBOL
- BPXBATCH — USS commands
- SYSTSIN / SYSTSPRT — in / out

Compile, Link & Go

IGYWCL / IGYWCLG procs · SYSLIB copybooks · SYSLMOD load library · binder (link-edit) · compile listing in SYSPRINT · DB2 precompile & BIND note

BUILDING A COBOL PROGRAM

A COBOL build has two steps: the **compiler** (IGYCRCTL) turns source into an object deck, and the **binder** (IEWL/IEWBLINK) links it into a **load module** stored in a load library. IBM procs combine them: **IGYWC** compiles, **IGYWCL** compiles and links, **IGYWCLG** also runs the program.

KEY POINTS

- Proc step names: COBOL (compile), LKED (link), GO (run)
- Compile listing and messages go to COBOL.SYSPRINT
- The load module name is the member name in SYSLMOD
- Compile RC 4 = warnings; 8+ = errors, so the link is skipped

IN COBOL

- `COPY member` statements are resolved from `COBOL.SYSLIB`
- Static CALLs are resolved by the binder from `LKED.SYSLIB`
- DB2 programs also need the SQL coprocessor option and a BIND

COMPILE AND LINK WITH IGYWCL

```
//USER01C  JOB (ACCT),'COMPILE',CLASS=A,MSGCLASS=X,
//         NOTIFY=&SYSUID
//         JCLLIB ORDER=(IGY.SIGYPROC)
//CL       EXEC IGYWCL,
//         PARM.COBOL='LIST,MAP,XREF,SSRANGE'
//COBOL.SYSIN   DD DSN=USER.COBOL.SRC(PAYCALC),DISP=SHR
//COBOL.SYSLIB  DD DSN=USER.COPYLIB,DISP=SHR
//              DD DSN=PROD.COPYLIB,DISP=SHR
//LKED.SYSLMOD  DD DSN=USER.LOADLIB(PAYCALC),DISP=SHR
//LKED.SYSLIB   DD DSN=CEE.SCEELKED,DISP=SHR
//              DD DSN=USER.LOADLIB,DISP=SHR
```

CODE EXPLAINED

- → JCLLIB points to the library holding the IGY procs (name varies by site)
- → `PARM.COBOL` sets compiler options for this compile
- → Copybooks are searched in USER.COPYLIB, then PROD.COPYLIB
- → The load module PAYCALC is written to USER.LOADLIB

MODERN EQUIVALENT

- ↔ **Python** No build: `python paycalc.py`; packaging with `pip wheel`
- ↔ **Java** `javac` compiles, `jar` packages (Maven/Gradle run both)
- ↔ **JavaScript** `tsc` compiles, a bundler (Vite/esbuild) links

ABEND CHECK

- ✖ Compile RC 8/12: read the `IGYxxxxx-E/S` messages in COBOL.SYSPRINT
- ✖ `IEW2456E SYMBOL … UNRESOLVED`: a statically called program is not in LKED.SYSLIB

IBM COBOL PROCS

PROC	STEPS
IGYWC	Compile
IGYWCL	Compile + link
IGYWCLG	Compile + link + go
IGYWCG	Compile + load and go

KEY DDS

DD	PURPOSE
COBOL.SYSIN	Source member
COBOL.SYSLIB	Copybooks
COBOL.SYSPRINT	Listing
LKED.SYSLMOD	Output load library
LKED.SYSLIB	Link libraries (LE, subprograms)
GO.SYSOUT	DISPLAY output (CLG)

⚠ COMMON MISTAKES

- No SYSLIB, so every COPY fails with severe errors
- Writing the load module to production by mistake
- Ignoring RC 4 warnings that point at real truncation bugs

✓ TIPS & BEST PRACTICES

- Keep one compile JCL member per program, or one proc with the name as a symbol
- Save listings with LIST/OFFSET for abend analysis
- Use IGYWCLG while learning so compile and run happen in one job

QUICK SUMMARY

- IGYCRCTL — compiler
- binder — makes load module
- IGYWCL / IGYWCLG — IBM procs
- SYSLIB / SYSLMOD — in / out

Sheet 25

Running a COBOL Batch Program

EXEC PGM= · STEPLIB · DD names ↔ SELECT … ASSIGN · PARM= → LINKAGE SECTION · DISPLAY → SYSOUT · RETURN-CODE → step RC

WIRING A PROGRAM TO ITS DATA

Running a COBOL program means one EXEC step plus a DD for **every file the program opens**. The DD names must match the `ASSIGN TO` names in the program. `PARM` arrives in the LINKAGE SECTION, DISPLAY output lands in SYSOUT, and the program's RETURN-CODE becomes the step's RC.

KEY POINTS

- Read the program's SELECT statements to know which DDs it needs
- Missing DD → FILE STATUS 35 on OPEN (or an abend)
- Add CEEDUMP so abends produce a useful dump
- Test and production differ only in DSNs and STEPLIB

IN COBOL

- `SELECT TRAN-IN ASSIGN TO TRANIN` ↔ `//TRANIN DD DSN=…`
- `PROCEDURE DIVISION USING LK-PARM` receives `PARM='2026-11'` as length + text
- `MOVE 4 TO RETURN-CODE` → step RC 4 → IF/COND in the JCL

A COMPLETE RUN STEP

```
//USER01R  JOB (ACCT),'PAYROLL RUN',CLASS=A,MSGCLASS=X,
//         NOTIFY=&SYSUID
//PAYRUN   EXEC PGM=PAYCALC,PARM='2026-11'
//STEPLIB  DD DSN=USER.LOADLIB,DISP=SHR
//* Files the program opens (match SELECT ... ASSIGN TO)
//TRANIN   DD DSN=PROD.PAY.TRANS.NOV,DISP=SHR
//ACCTMAST DD DSN=PROD.ACCT.KSDS,DISP=SHR
//PAYOUT   DD DSN=USER.PAY.OUT,DISP=(NEW,CATLG,DELETE),
//            SPACE=(CYL,(5,5),RLSE),RECFM=FB,LRECL=150
//RPTOUT   DD SYSOUT=*
//REJECTS  DD DSN=USER.PAY.REJECTS,DISP=(NEW,CATLG,DELETE),
//            SPACE=(TRK,(5,5),RLSE),RECFM=FB,LRECL=180
//* Runtime DDs
//SYSOUT   DD SYSOUT=*
//CEEDUMP  DD SYSOUT=*
//SYSUDUMP DD SYSOUT=*
//         IF (PAYRUN.RC > 4) THEN
//ALERT    EXEC PGM=IEFBR14
//         ENDIF
```

CODE EXPLAINED

- → Five file DDs match five SELECT statements in PAYCALC
- → PARM '2026-11' is the run period
- → SYSOUT collects DISPLAY messages and control totals
- → The IF step reacts to RC 8 or higher from PAYCALC

MODERN EQUIVALENT

- ↔ **Python** `python paycalc.py 2026-11` with paths in env vars; `sys.exit(rc)`
- ↔ **Java** `java -cp app.jar PayCalc 2026-11` with config; `System.exit(rc)`
- ↔ **JavaScript** `node paycalc.js 2026-11` with `process.env` paths; `process.exitCode = rc`

ABEND CHECK

- ✖ FILE STATUS 35 / `IEC130I … DD STATEMENT MISSING`: a SELECT's DD name is not in the step
- ✖ S806: STEPLIB does not contain PAYCALC (or the name is misspelled)

PROGRAM ↔ JCL

COBOL	JCL
`ASSIGN TO TRANIN`	//TRANIN DD
`FD record length`	LRECL
`RECORDING MODE F`	RECFM=FB
`PROCEDURE DIVISION USING`	PARM=
`DISPLAY`	//SYSOUT DD
`ACCEPT`	//SYSIN DD
`RETURN-CODE`	Step RC

⚠ COMMON MISTAKES

- A DD name typo that only shows up as status 35 at run time
- Output LRECL not matching the FD record
- No CEEDUMP DD, so an abend leaves little to debug

✓ TIPS & BEST PRACTICES

- List the program's DD names in its header comment
- Use symbols for the environment qualifier so one JCL fits test and prod
- Check control totals in SYSOUT after every run

QUICK SUMMARY

- one DD per SELECT
- `PARM` → LINKAGE
- `DISPLAY` → SYSOUT
- RETURN-CODE → step RC

Sheet 26

Generation Data Groups (GDGs)

DEFINE GDG · LIMIT / SCRATCH / NOEMPTY · relative generations (+1) / (0) / (−1) · G0001V00 names · daily-file pattern

VERSIONED DATASETS

A **GDG** is a group of datasets with one base name, each run adding a new **generation**. Programs refer to generations by relative number: (0) is the newest, (−1) the one before, (+1) a new one created in this job. The base keeps only the last LIMIT generations, rolling older ones off automatically.

KEY POINTS

- Real names end in .GnnnnV00 (e.g. .G0042V00)
- Relative numbers are fixed for the whole job
- The base is defined once with IDCAMS
- Giving the base name alone reads all generations, newest first

DEFINE AND USE A GDG

```
//* One-time setup: keep the last 7 generations
//DEFGDG   EXEC PGM=IDCAMS
//SYSPRINT DD SYSOUT=*
//SYSIN    DD *
  DEFINE GDG (NAME(PROD.SALES.DAILY) -
         LIMIT(7) NOEMPTY SCRATCH)
/*
//*
//* Daily job: read yesterday, write today
//DAILY    EXEC PGM=SALESUPD
//PREVDAY  DD DSN=PROD.SALES.DAILY(0),DISP=SHR
//NEWDAY   DD DSN=PROD.SALES.DAILY(+1),
//            DISP=(NEW,CATLG,DELETE),
//            SPACE=(CYL,(5,5),RLSE),RECFM=FB,LRECL=100
//*
//* Later step in the SAME job sees the new one as (+1)
//REPORT   EXEC PGM=SALESRPT
//SALESIN  DD DSN=PROD.SALES.DAILY(+1),DISP=SHR
//*
//* Read every generation at once
//ALLDAYS  DD DSN=PROD.SALES.DAILY,DISP=SHR
```

CODE EXPLAINED

- → LIMIT(7): the eighth generation pushes the oldest out
- → SCRATCH deletes rolled-off generations; NOEMPTY rolls off one at a time
- → (0) at job start is yesterday's file; (+1) is today's
- → Inside the same job the new generation stays (+1)

MODERN EQUIVALENT

- ↔ **Python** Date-stamped files sales_2026-11-12.csv plus a cleanup that keeps 7
- ↔ **Java** Rolling files (Logback maxHistory=7)
- ↔ **JavaScript** Rotating logs (rotating-file-stream with maxFiles: 7)

ABEND CHECK

- ✖ Allocation fails on (+1): the GDG base was never defined; run the IDCAMS DEFINE GDG first
- ✖ Rerun creates another (+1) and shifts (0): restart carefully or use absolute names

RELATIVE REFERENCES

REFERENCE	MEANS
(+1)	New generation (this job)
(0)	Newest existing
(−1)	One before newest
(−n)	n before newest
base name only	All generations

DEFINE GDG OPTIONS

OPTION	EFFECT
LIMIT(n)	Generations kept (max 255; 999 extended)
SCRATCH / NOSCRATCH	Delete / keep rolled-off
EMPTY / NOEMPTY	Roll all / one at limit
PURGE	Allow delete before expiry

⚠ COMMON MISTAKES

- Expecting (0) to mean the generation created earlier in the same job (it is still +1)
- Rerunning a failed job and creating an extra generation
- Using EMPTY, which uncatalogs all generations when the limit is hit

✓ TIPS & BEST PRACTICES

- Use NOEMPTY SCRATCH unless you have a reason not to
- Note in run books how to rerun safely (delete the bad +1 first)
- Use GDGs for daily, weekly and monthly files instead of date-named datasets

QUICK SUMMARY

- GDG — versioned dataset
- (+1) new · (0) newest · (−1) previous
- LIMIT — generations kept
- relative numbers — fixed per job

JES2 & Job Output

WHERE JOBS AND OUTPUT GO

JES2 reads submitted jobs, queues them by **class**, starts them on an initiator and stores their output on the **spool**. Each job has three system files: **JESMSGLG** (job log and step summary), **JESJCL** (the expanded JCL) and **JESYSMSG** (allocation messages and condition codes), followed by your SYSOUT DDs.

KEY POINTS

- Job IDs look like `JOB04711`
- Phases: input → conversion → execution → output → purge
- MSGCLASS decides whether output is held or printed
- Held output stays until you purge it or it ages off

READING OUTPUT WITH ZOWE AND SDSF

```
# List my jobs
zowe jobs list jobs --owner USER01
# JOB04711  PAYROLL  OUTPUT  CC 0008

# List the job's spool files
zowe jobs list sfbi JOB04711
# 1 JESMSGLG  2 JESJCL  3 JESYSMSG  101 SYSOUT  102 RPTOUT

zowe jobs view sfbi JOB04711 3     # condition codes per step
zowe jobs view sfbi JOB04711 101   # program DISPLAY output

# SDSF equivalents (ISPF)
#   =SD ; ST         status panel
#   ?  next to job   list spool files
#   S  next to file  view it
#   P  next to job   purge
```

CODE EXPLAINED

→ CC 0008 is the highest step return code in the job

→ JESYSMSG lists each step with its condition code

→ Your program's DISPLAY output is in its SYSOUT DD file

→ Purge output you no longer need to free spool space

MODERN EQUIVALENT

↔ **Python** CI job logs per step (GitHub Actions run page)

↔ **Java** Spring Batch job repository and logs

↔ **JavaScript** Process stdout/stderr captured by a job runner or PM2

ABEND CHECK

✖ No output at all: MSGCLASS sends it to a printer or discards it; use a held class

✖ `$HASP…` messages in JESMSGLG: JES2 problems such as spool full or job held

SPOOL FILES

FILE	CONTAINS
JESMSGLG	Job log, step summary
JESJCL	Expanded JCL, procs
JESYSMSG	Allocation, CCs, IEF messages
SYSOUT / SYSPRINT	Program / utility output
CEEDUMP	LE dump on abend

JOB STATUS

STATUS	MEANING
INPUT	Queued or held
ACTIVE	Running
OUTPUT	Done, output on spool
CC nnnn	Highest RC
ABEND Sxxx / Unnnn	Abnormal end
JCL ERROR	Never ran

⚠ COMMON MISTAKES

- Purging output before reading why the job failed
- Looking only at the last file instead of JESYSMSG's step codes
- Submitting to a class that is not started, so the job waits forever

✓ TIPS & BEST PRACTICES

- Check JESYSMSG first: it shows every step's RC or abend
- Download output with Zowe to attach it to tickets
- Ask which classes your site uses for test jobs and held output

QUICK SUMMARY

- JES2 — queue, run, spool
- JESMSGLG / JESJCL / JESYSMSG
- CC — highest RC
- SDSF / Zowe — view output

Sheet 28

Reading Job Logs & JCL Errors

JCL ERROR · IEFC / IEF messages · IEF212I dataset not found · IEF453I job failed · IEC open errors · dataset in use (enqueue)

FINDING WHAT WENT WRONG

A **JCL error** means z/OS rejected the job before any step ran: a syntax problem, a missing dataset or a bad parameter. The message ID tells you where to look: **IEFC** messages are syntax errors, **IEF** messages are allocation and step messages, and **IEC** messages are open/close errors from the access methods.

KEY POINTS

- Message IDs end in a severity: I info, W warning, E/A error/action
- JESJCL shows the line number the message refers to
- Datasets in use: the job waits with an enqueue message
- `TYPRUN=SCAN` catches syntax errors before running

A FAILED JOB'S JESYSMSG

```
 STMT NO. MESSAGE
       5 IEFC605I UNIDENTIFIED OPERATION FIELD
IEF212I USER01R PAYRUN TRANIN - DATA SET NOT FOUND
IEF272I USER01R PAYRUN - STEP WAS NOT EXECUTED.
IEF453I USER01R - JOB FAILED - JCL ERROR

IEC141I 013-18,IGG0191B,USER01R,PAYRUN,SRCIN,
        USER.COBOL.SRC(PAYCLAC)
IEF450I USER01R PAYRUN - ABEND=S013 U0000 REASON=00000018

IEF861I FOLLOWING RESERVED DATA SET NAMES UNAVAILABLE
IEF863I DSN = PROD.ACCT.MASTER  USER01R RC = 04
IEF099I JOB USER01R WAITING FOR DATA SETS
```

CODE EXPLAINED

→ Statement 5 has a misspelled operation (e.g. `EXCE` for EXEC)

→ IEF212I: the TRANIN dataset name is wrong or not cataloged

→ 013-18: member PAYCLAC not found (typo for PAYCALC)

→ IEF861I/IEF099I: another job holds PROD.ACCT.MASTER with DISP=OLD

MODERN EQUIVALENT

↔ **Python** A Python traceback: the last line names the error, the file and line

↔ **Java** A Java stack trace: exception type, message, then the call chain

↔ **JavaScript** A Node error: `ENOENT: no such file or directory, open '…'`

ABEND CHECK

✖ Reading only the last message: the first error usually caused the rest

✖ Job waits forever on IEF099I: find the job holding the dataset (SDSF `ENQ`) or use DISP=SHR

MESSAGE PREFIXES

PREFIX	SOURCE
IEFC	JCL conversion (syntax)
IEF	Allocation / initiator
IEC	Data management (OPEN)
IGD	SMS
IDC	IDCAMS
IGY	COBOL compiler
CEE	Language Environment
$HASP	JES2

COMMON MESSAGES

MESSAGE	MEANING
IEFC605I	Bad operation field
IEFC621I	Continuation expected
IEF212I	Dataset not found
IEF287I NOT CATLGD 2	Name already cataloged
IEF453I	Job failed, JCL error
IEC141I 013-18	Member not found
IEC130I	DD statement missing

⚠ COMMON MISTAKES

- Fixing a later error message when the first one caused it
- Ignoring warnings like NOT CATLGD that cause next-day failures
- Resubmitting unchanged JCL hoping it works this time

✓ TIPS & BEST PRACTICES

- Search the IBM docs for the full message ID
- Use TYPRUN=SCAN after every JCL edit
- Keep a personal list of the messages you meet most often

QUICK SUMMARY

- IEFC — syntax
- IEF212I — dataset not found
- IEC141I 013-18 — member missing
- first message first

Common Abends

ABEND CODES YOU WILL SEE

An **abend** ends a step abnormally. **System** abends (Sxxx) come from z/OS or the hardware; **user** abends (Unnnn) are raised by programs or Language Environment. The code plus the reason code and the CEEDUMP usually point straight at the cause.

KEY POINTS

- Abended steps have no RC; later steps are bypassed unless COND=EVEN/ONLY or `IF ABEND`
- Reason codes add detail: S013-18, S806-04
- SOCx abends are program checks (bad data or address)
- LE turns many program checks into U4038 plus CEE messages

ABEND IN THE JOB LOG

```
IEA995I SYMPTOM DUMP OUTPUT
  SYSTEM COMPLETION CODE=0C7  REASON CODE=00000007
   TIME=02.14.31  SEQ=00412  CPU=0000  ASID=0031
   PSW AT TIME OF ERROR  078D1000 8001A6E2
     ACTIVE LOAD MODULE           ADDRESS=0001A000
     NAME=PAYCALC
     OFFSET=000006E2
IEF450I USER01R PAYRUN - ABEND=S0C7 U0000 REASON=00000007

CEE3207S The system detected a data exception (System
         Completion Code=0C7).
         From compile unit PAYCALC at entry point PAYCALC at
         statement 214 at compile unit offset +000006E2
```

CODE EXPLAINED

→ S0C7: a packed/zoned decimal instruction hit invalid data

→ The OFFSET maps to a statement in the LIST/OFFSET compile listing

→ The CEE message gives the statement number directly (with TEST/listing info)

→ Look at the input record being processed at the time

MODERN EQUIVALENT

↔ **Python** `decimal.InvalidOperation` or `ValueError` with a traceback line number

↔ **Java** `NumberFormatException` / `ArithmeticException` with a stack trace

↔ **JavaScript** `NaN` results or a `RangeError` / `TypeError` with a stack trace

ABEND CHECK

✖ S0C7 at 2 a.m.: find the bad record (key in SYSOUT), fix or skip it, rerun from the step

✖ S806 after a deploy: the new load module was not copied to the library in STEPLIB

SYSTEM ABENDS

CODE	USUAL CAUSE
S0C1	Operation exception (bad branch)
S0C4	Protection exception (bad address)
S0C7	Data exception (bad numeric)
S0CB	Decimal divide by zero
S013	OPEN failed (DCB, member)
S106	Load module fetch error
S222	Cancelled by operator/user
S322	CPU TIME exceeded
S806	Module not found
S878 / S80A	Not enough storage (REGION)
SB37 / SD37 / SE37	Out of space

USER ABENDS

CODE	MEANING
U4038	LE severe error (see CEE msg)
U4039	LE abend with dump
U4093	LE initialization error
U0016 etc.	Program-defined (CEE3ABD)

⚠ COMMON MISTAKES

- Rerunning an S0C7 job without fixing the data
- Raising REGION for an S0C4 (it is a bad address, not memory size)
- Raising TIME for an S322 caused by an infinite loop

✓ TIPS & BEST PRACTICES

- Always keep CEEDUMP and SYSUDUMP DDs in COBOL steps
- Log record counts and keys so the bad record is easy to find
- Map offsets with the LIST/OFFSET listing of the exact module version

QUICK SUMMARY

- S0C7 — bad numeric data
- S0C4 — bad address
- S806 — module not found
- S322 / SB37 — time / space

Sheet 30

Scheduling & Modern Workflow

batch windows · schedulers (IBM Workload Scheduler, Control-M) · job dependencies · JCL in Git · `zowe zos-jobs submit … --wait-for-output` in CI · z/OSMF REST jobs API

FROM SUBMITTING BY HAND TO PIPELINES

Production jobs are run by a **scheduler** such as IBM Workload Scheduler or Control-M, which starts jobs at set times or when other jobs finish, and alerts on failures. Development teams now keep JCL in **Git** and use **Zowe CLI** or the **z/OSMF REST API** to submit jobs and check return codes from CI pipelines.

KEY POINTS

- Schedulers track dependencies: job B runs after job A ends RC ≤ 4
- The **batch window** is the overnight period jobs must finish in
- Restarts are run from the scheduler, usually with RESTART=step
- CI should fail the build when a job's RC is above 4 or it abends

SUBMITTING FROM GITHUB ACTIONS

```
# .github/workflows/zos.yml (run step)
npm install -g @zowe/cli
zowe config set profiles.zosmf.properties.host "$ZOS_HOST"

# Upload JCL and source from the repo
zowe files upload dir-to-pds jcl/ "USER.CI.JCL"
zowe files upload dir-to-pds cobol/ "USER.CI.COBOL"

# Compile, then run tests; fail on RC > 4 or abend
for job in COMPILE TESTRUN; do
  out=$(zowe jobs submit ds "USER.CI.JCL($job)" \
        --wait-for-output --rfj)
  rc=$(echo "$out" | jq -r '.data.retcode')
  echo "$job: $rc"
  case "$rc" in "CC 0000"|"CC 0004") ;; *) exit 1 ;; esac
done

# z/OSMF REST: PUT /zosmf/restjobs/jobs  (submit)
#              GET /zosmf/restjobs/jobs/{name}/{id}  (status)
```

CODE EXPLAINED

→ `--rfj` returns JSON so the script can read the retcode

→ RC 0 and 4 pass; 8, abends and JCL errors fail the pipeline

→ Uploading from Git makes the repo the source of truth

→ The same API calls work from Jenkins or GitLab CI

MODERN EQUIVALENT

↔ **Python** Airflow DAGs or cron + a Python driver that checks exit codes

↔ **Java** Spring Batch or Quartz jobs with dependency flows

↔ **JavaScript** GitHub Actions / node-cron with `needs:` dependencies

ABEND CHECK

✖ CI passes but the job abended: you checked "submitted" instead of the final retcode

✖ Scheduler shows the job late: an upstream job failed and dependents are waiting

SCHEDULERS

PRODUCT	NOTES
IBM Workload Scheduler (TWS / OPC)	IBM's z/OS scheduler
Control-M (BMC)	Cross-platform
CA 7 (Broadcom)	z/OS scheduler
Zowe + CI	Dev/test pipelines

RETCODE VALUES

RETCODE	MEANING
CC 0000	Success
CC 0004	Warnings
CC 0008+	Errors
ABEND S0C7	Abended
JCL ERROR	Did not run
null	Still running / not found

⚠ COMMON MISTAKES

- Pipelines that do not wait for job completion
- Hard-coded passwords in workflow files (use repository secrets)
- Editing production JCL directly instead of through Git and the scheduler

✓ TIPS & BEST PRACTICES

- Store host and credentials as CI secrets
- Name CI datasets per branch or build to avoid collisions
- Download job output as build artifacts for every run

QUICK SUMMARY

- scheduler — times + dependencies
- batch window — overnight limit
- Zowe `--wait-for-output` — CI submit
- fail CI on RC > 4 or abend

Sheet 31

Quick Reference — Statements & Parameters

JOB · EXEC · DD · DISP combos · SPACE / DCB · special DD names · IF / COND

JOB (06)

PARAMETER	MEANING
`(acct),'name'`	Accounting, programmer
`CLASS=`	Job class
`MSGCLASS=`	Log output class
`MSGLEVEL=(1,1)`	Full listing
`NOTIFY=&SYSUID`	Notify submitter
`REGION= / TIME=`	Memory / CPU
`TYPRUN=SCAN / HOLD`	Check / hold
`RESTART=step`	Rerun from step

EXEC (07)

PARAMETER	MEANING
`PGM=name`	Run program
`procname`	Run procedure
`PARM='…'`	≤100 chars
`PARMDD=dd`	Long PARM
`COND=(n,op)`	Skip test
`REGION= / TIME=`	Step limits
`PARM.pstep=`	Proc override

DD (08–13)

PARAMETER	MEANING
`DSN=`	Dataset name
`DISP=`	Status, normal, abnormal
`SPACE=`	Size
`RECFM= LRECL=`	Record shape
`BLKSIZE=0`	System block size
`LIKE=`	Copy attributes
`SYSOUT=*`	Job log
`DUMMY`	Empty / discard
`DD * / DATA`	In-stream
`DSN=&&T`	Temp dataset

DISP (09)

CODING	USE
`SHR`	Read, shared
`OLD`	Exclusive
`MOD`	Append
`(NEW,CATLG,DELETE)`	Create
`(NEW,PASS)`	Temp to next step
`(OLD,DELETE)`	Delete after use
`(MOD,DELETE)`	Safe delete

SPACE & DCB (10, 11)

CODING	MEANS
`SPACE=(TRK,(p,s))`	Tracks
`SPACE=(CYL,(p,s,d))`	Cylinders + dir
`,RLSE`	Release unused
`AVGREC=K`	Record units
`RECFM=FB / VB / FBA / U`	Format
`DSNTYPE=LIBRARY`	PDSE

SPECIAL DDS (12)

DD	USE
`JOBLIB / STEPLIB`	Program libraries
`SYSIN`	Control input
`SYSOUT / SYSPRINT`	Messages
`CEEDUMP / SYSUDUMP`	Dumps
`SYSTSIN / SYSTSPRT`	TSO batch
`SYSLIB`	Copybooks / links

CONDITIONS (14, 15)

CODING	EFFECT
`// IF (RC <= 4) THEN`	Run if OK
`// IF (STEP1.RC = 0) THEN`	One step
`// IF (ABEND) THEN`	After abend
`// ELSE / // ENDIF`	Branch / end
`COND=(4,LT)`	Skip if RC > 4
`COND=EVEN / ONLY`	Abend handling

PROCS & SYMBOLS (16–18)

CODING	EFFECT
`JCLLIB ORDER=(lib)`	Proc / include libs
`//P PROC X=dflt`	Define
`// PEND`	End in-stream
`// SET X=val`	Symbol
`&X..`	Symbol + period
`INCLUDE MEMBER=m`	Paste JCL
`//pstep.dd DD …`	Override

SYNTAX (05)

ITEM	RULE
`//`	Columns 1–2
`Name`	Column 3, ≤8 chars
`Continue`	End with comma; resume col 4–16
`//*`	Comment
`/*`	End data
`Column 72`	Last usable is 71

Sheet 32

Quick Reference — Utilities, RCs & Abends

utility → DD names → control statements · return codes · abend codes · common IEF / IEC messages

UTILITIES (19–24)

PROGRAM	DDS	DOES
IEFBR14	any	Allocate / delete via DD
IEBGENER	SYSUT1, SYSUT2, SYSIN	Copy / print
IEBCOPY	INDD, OUTDD, SYSIN	Library copy, compress
IDCAMS	SYSIN, SYSPRINT	VSAM, GDG, catalog
SORT	SORTIN, SORTOUT, SYSIN, SYSOUT	Sort, filter, reformat
IKJEFT01	SYSTSIN, SYSTSPRT	TSO / REXX / DB2
BPXBATCH	STDOUT, STDERR	USS commands
IGYWCL	COBOL.SYSIN, LKED.SYSLMOD	Compile + link

CONTROL STATEMENTS

STATEMENT	UTILITY
COPY OUTDD=o,INDD=i	IEBCOPY
SELECT MEMBER=(…)	IEBCOPY
DEFINE CLUSTER / GDG	IDCAMS
REPRO / PRINT / LISTCAT	IDCAMS
SET MAXCC=0	IDCAMS
SORT FIELDS=(p,l,f,A)	SORT
INCLUDE COND=(…)	SORT
OUTREC BUILD=(…)	SORT
SUM FIELDS=NONE	SORT

RETURN CODES (14)

RC	MEANING
0	Success
4	Warning
8	Error
12	Severe
16	Fatal / sort failed
FLUSH	Step bypassed
JCL ERROR	Job never ran

GDG (26)

REF	MEANS
(+1)	New
(0)	Newest
(-1)	Previous
base	All generations
LIMIT(n)	Kept

ABENDS (29)

CODE	CAUSE
S0C4	Bad address
S0C7	Bad numeric data
S0CB	Divide by zero
S013	OPEN / DCB / member
S222	Cancelled
S322	CPU time
S806	Module not found
S878 / S80A	Storage
SB37 / SD37 / SE37	Space
U4038	LE error

MESSAGES (28)

MESSAGE	MEANING
IEFC605I	Bad operation
IEF212I	Dataset not found
IEF287I … 2	Already cataloged
IEF453I	JCL error
IEC130I	DD missing
IEC141I 013-18	Member not found
IEC141I 013-20	BLKSIZE / LRECL
IEF099I	Waiting for dataset